

目录

1 深度学习框架完整搭建流程	3
1.1 检查环境配置	3
1.2 确定自己需要安装的 pytorch 版本	3
1.3 安装 cuda	4
1.4 安装 cudann	6
1.5 环境变量的介绍	8
1.6 cuda 版本的替换	10
1.7 anaconda 的安装（pycharm 的安装）	12
1.8 anaconda 的路径设置（下载包会比较快）	16
1.9 虚拟环境的创建	16
1.10 虚拟环境的使用（jupyter）	18
1.11 虚拟环境的使用（pycharm）	23
1.12 安装 pytorch 框架	26
2 各种代码问题总结	27
2.1 python 自己包的配置命令	27
2.2 Windows 的 pyG 安装方式	27
2.3 R 相关配置	27
2.4 项目路径设置	27
2.5 linux 命令总结	28
2.5.1 文件与目录操作（7 个）	29
2.5.2 文件内容查看与处理（5 个）	31
2.5.3 系统管理与状态查看（7 个）	31
2.5.4 用户与权限管理（5 个）	32
2.5.5 网络操作（4 个）	33
2.5.6 压缩与解压（3 个）	33
2.5.7 周总提供	33
2.6 Python→requirements 文件	34
2.7 VS 连接远程服务器	34
2.8 linux 配置 rpy2 模块	39
2.9 github 项目包配置（setup.py 方法）	41
2.10 虚拟环境调试	47
2.11 已经安装了某个包 import 还是报错解决方案	48
2.12 去修改 github 源码解决报错	50
2.13 生存分析	54
2.14 包的某个依赖已经被弃用解决方法	55

2.15 Sphinx+github+ReadtheDocs 搭建自己的代码 Tutorials	56
2.15.1 Sphinx 部分	56
2.15.2 github 部分	61
2.15.3 ReadtheDocs 部分	64
2.15.4 如何去更新网站	69

1 深度学习框架完整搭建流程

Windows 操作系统、NVIDIA 显卡、c 盘内存空闲>10G

1.1 检查环境配置

“Windows+r”，输入“cmd”进入控制台输入：

nvidia-smi

```
C:\Users\18085>nvidia-smi
Sat Sep 14 23:28:11 2024
```

NVIDIA-SMI 556.13			Driver Version: 556.13			CUDA Version: 12.5		
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	NVIDIA GeForce RTX 4050	...	WDDM	00000000:01:00.0	Off		N/A	
N/A	35C	P3	11W / 70W	0MiB / 6141MiB		0%	Default	
							N/A	

```

Processes:
GPU  GI  CI  PID  Type  Process name  GPU Memory Usage
ID   ID   ID
=====
No running processes found

```

该电脑支持 cuda 的最高版本。

1.2 确定自己需要安装的 pytorch 版本

打开网址：

<https://pytorch.org/>

下滑找到你想要的那个版本，确定你可以安装的 cuda 版本

NOTE: Latest PyTorch requires Python 3.8 or later.

PyTorch Build	Stable (2.4.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	CUDA 12.4	ROCm 6.1
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

Previous versions of PyTorch >

蓝色箭头就是你的安装指令，红色方框是更早的版本。

现在只是确定 pytorch 版本，请不要安装 pytorch!!!

1.3 安装 cuda

进入 cuda 官网：

<https://developer.nvidia.com/cuda-toolkit-archive>

Latest Release
[CUDA Toolkit 12.6.1 \(August 2024\), Versioned Online Documentation](#)

Archived Releases
[CUDA Toolkit 12.6.0 \(August 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.5.1 \(July 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.5.0 \(May 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.4.1 \(April 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.4.0 \(March 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.3.2 \(January 2024\), Versioned Online Documentation](#)
[CUDA Toolkit 12.3.1 \(November 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.3.0 \(October 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.2.2 \(August 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.2.1 \(July 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.2.0 \(June 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.1.1 \(April 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.1.0 \(February 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.0.1 \(January 2023\), Versioned Online Documentation](#)
[CUDA Toolkit 12.0.0 \(December 2022\), Versioned Online Documentation](#)
[CUDA Toolkit 11.8.0 \(October 2022\), Versioned Online Documentation](#)
[CUDA Toolkit 11.7.1 \(August 2022\), Versioned Online Documentation](#)
[CUDA Toolkit 11.7.0 \(May 2022\), Versioned Online Documentation](#)
[CUDA Toolkit 11.6.2 \(March 2022\), Versioned Online Documentation](#)
[CUDA Toolkit 11.6.1 \(February 2022\), Versioned Online Documentation](#)

选择自己想要的 cuda 版本，点击，选择自己的系统（推荐 local 版本），下载。

Operating System

Linux Windows

Architecture

x86_64

Version

10 11 Server 2016 Server 2019 Server 2022

Installer Type

exe (local) exe (network)

Download Installer for Windows 11 x86_64

The base installer is available for download below.

> Base Installer

Download (3.0 GB)

安装路径可以自行选择，安装选项“自定义”→后续都勾选。



在安装 CUDA 工具包时，“Visual Studio Integration”选项是为了让 CUDA 与 Visual Studio 集成，这样可以在 Visual Studio 中更方便地创建、编译和调试 CUDA 项目。是否勾选取决于具体需求。如果不确定，那么默认勾选通常是一个安全的选择，尤其是对于初学者来说，这样可以简化开发流程。

一般第一次安装后会要你重启电脑。

打开控制台，输入：

```
nvcc -V
```

就可以看到安装的版本了。

```
C:\Users\18085>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Sep_21_10:41:10_Pacific_Daylight_Time_2022
Cuda compilation tools, release 11.8, V11.8.89
Build cuda_11.8.r11.8/compiler.31833905_0
```

这个时候就表明 cuda 安装成功了。

或者控制台输入：

```
echo %CUDA_PATH%
```

会返回你的安装路径

```
C:\Users\18085>echo %CUDA_PATH%  
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.8
```

1.4 安装 cudann

进入下载链接：

<https://developer.nvidia.com/cudnn-downloads>

不一定需要下载最新的，点击即可进入老版本

cuDNN 9.4.0 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree and conditions of the [NVIDIA Software License Agreement](#).

Operating System

Linux

Windows

Resources

- [cuDNN Documentation](#)
- [Tarball and Zip Archive Deliverables](#)
- [Archive of Previous Releases](#)

进入老版本：

Latest Release

[cuDNN 9.4.0](#) (September 2024), [Documentation](#)

Archived Releases

[cuDNN 9.3.0](#) (August 2024)
[cuDNN 9.2.1](#) (July 2024)
[cuDNN 9.2.0](#) (June 2024)
[cuDNN 9.1.1](#) (May 2024)
[cuDNN 9.1.0](#) (April 2024)
[cuDNN 9.0.0](#) (February 2024)
[cuDNN 8.x - 1.x](#) (December 2023 - August 2014)

查看其能够接受的 cuda 版本，后面的数字就表示 cudann 能够匹配的 cuda 版本：

cuDNN Archive

NVIDIA cuDNN is a GPU-accelerated library of primitives for deep neural networks.

[Download cuDNN v8.9.7 \(December 5th, 2023\), for CUDA 12.x](#)

[Download cuDNN v8.9.7 \(December 5th, 2023\), for CUDA 11.x](#)

[Download cuDNN v8.9.6 \(November 1st, 2023\), for CUDA 12.x](#)

[Download cuDNN v8.9.6 \(November 1st, 2023\), for CUDA 11.x](#)

[Download cuDNN v8.9.5 \(October 27th, 2023\), for CUDA 12.x](#)

[Download cuDNN v8.9.5 \(October 27th, 2023\), for CUDA 11.x](#)

[Download cuDNN v8.9.4 \(August 8th, 2023\), for CUDA 12.x](#)

[Download cuDNN v8.9.4 \(August 8th, 2023\), for CUDA 11.x](#)

选择所需要的版本，点击下载 Windows 版本：

Local Installers for Windows and Linux, Ubuntu(x86_64, armsbsa)

[Local Installer for Windows \(Zip\)](#)

[Local Installer for Linux x86_64 \(Tar\)](#)

[Local Installer for Linux PPC \(Tar\)](#)

[Local Installer for Linux SBSA \(Tar\)](#)

[Local Installer for Debian 11 \(Deb\)](#)

[Local Installer for Ubuntu18.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu22.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 aarch64sbsa \(Deb\)](#)

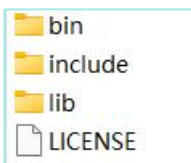
[Local Installer for Ubuntu22.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu20.04 cross-sbsa \(Deb\)](#)

[Local Installer for Ubuntu22.04 cross-sbsa \(Deb\)](#)

注意，此时下载是需要注册账号的，按照他的步骤进行注册即可。

下载下来的是一个安装包，解压，里面的文件有：



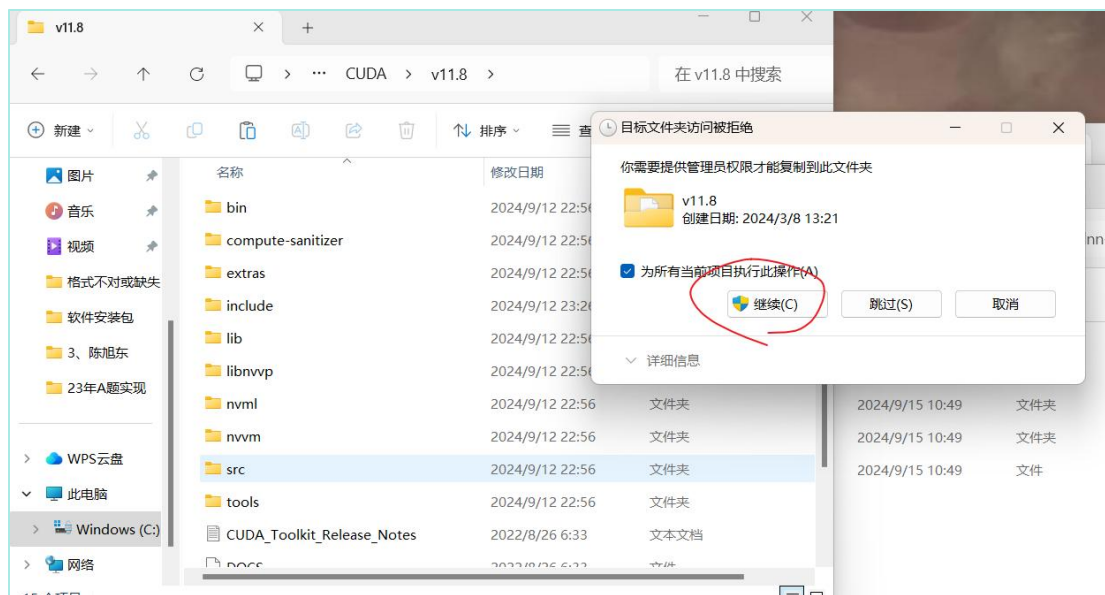
如果 cuda 装在默认路径，请将上述文件复制到路径 C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v（版本号）即可。如果是其他路径，请在控制台输入：

```
echo %CUDA_PATH%
```

会返回你的安装路径：

```
C:\Users\18085>echo %CUDA_PATH%
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.8
```

复制到该路径，并且覆盖原文件即可。



1.5 环境变量的介绍

Windows 的环境变量是操作系统中的一个功能，它存储了用于操作系统和应用程序运行时的一系列变量值。环境变量可以用来定义系统范围内的设置，如程序路径、系统文件位置、控制台窗口属性等。初学者一般遇到问题就是缺少部分环境变量，系统不知道去哪找。

路径设置（Path 变量）：PATH 环境变量列出了操作系统在执行命令时搜索可执行文件的位置。当你输入一个命令时，操作系统会按照 PATH 变量中的目录顺序查找对应的程序。

%SystemRoot%：Windows 操作系统的安装目录。

%ProgramFiles%：大多数程序默认安装的目录。

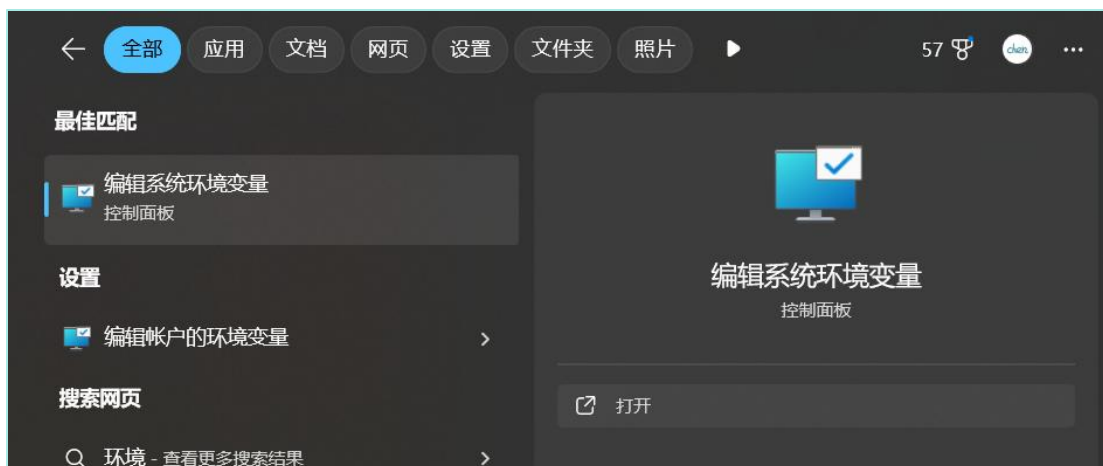
%APPDATA%：存储应用程序数据的目录。

%TEMP%或%TMP%：用于存储临时文件的目录。

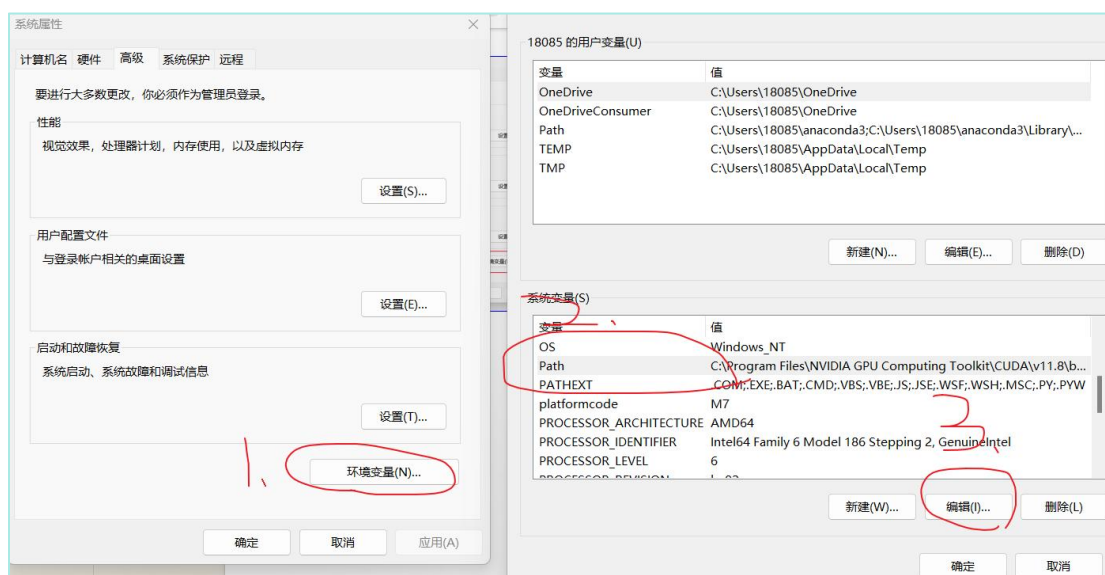
%PATH%：执行命令时要搜索的目录列表。

打开环境变量：

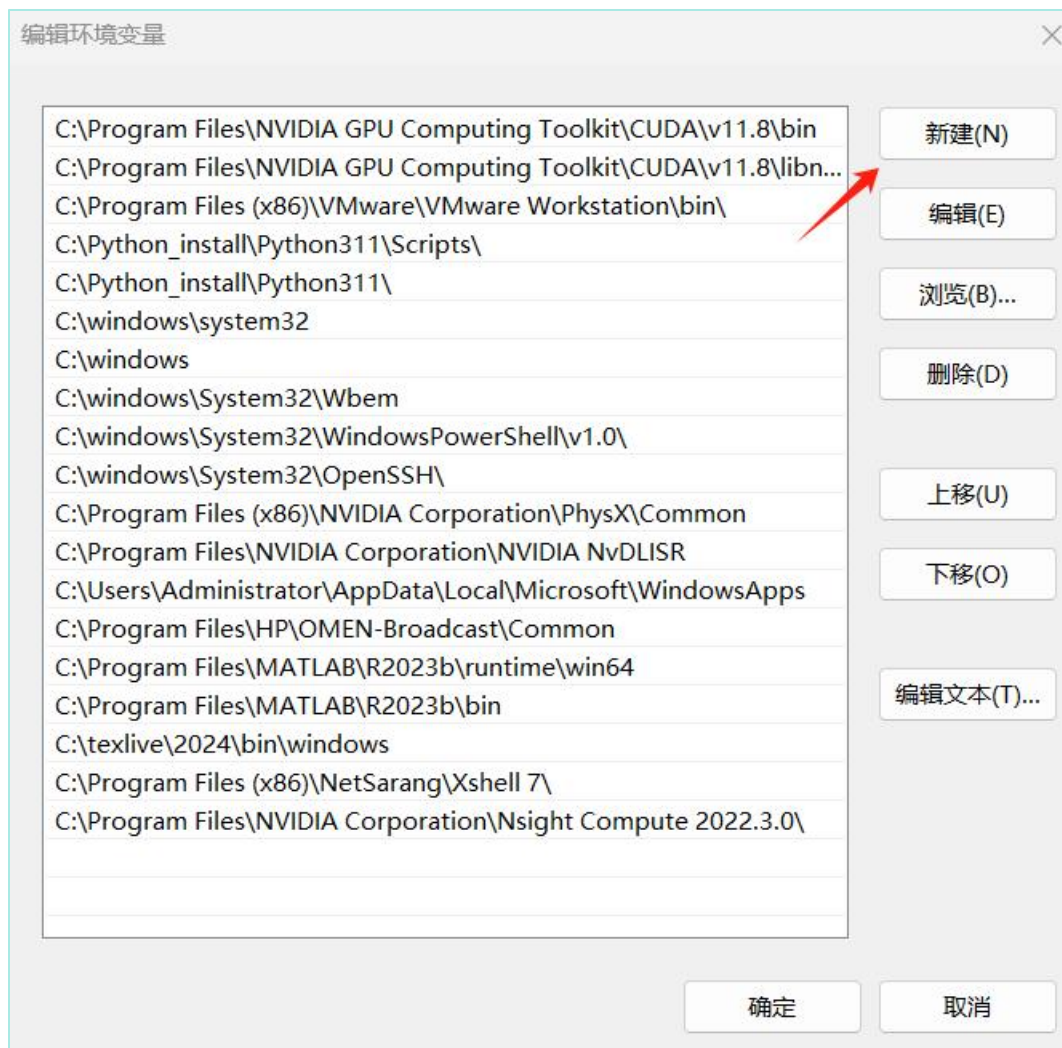
搜索“环境变量”，打开：



进入，点击红框：



点“新建”就可以添加环境变量了。



最后记得点击“确定”即添加成功。

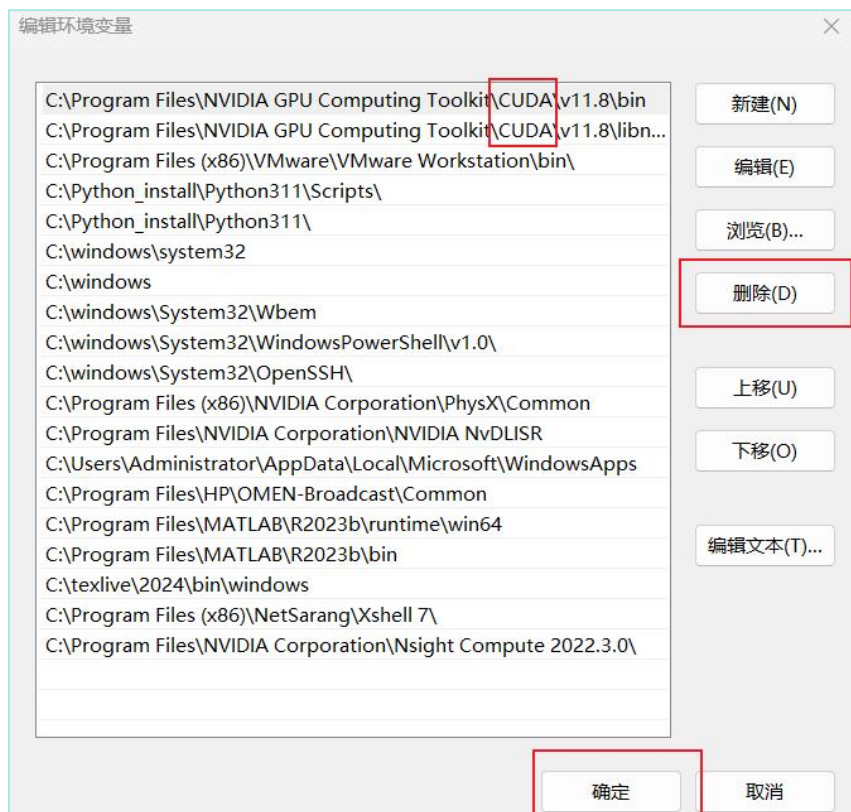
1.6 cuda 版本的替换

如果你发现你需要一个其他的 cuda 版本，以下介绍 cuda 的卸载（不太推荐安装多个 cuda，后面大项目一般都在服务器跑）。

1.在应用中搜索“cuda”，显示出来的全部卸载：

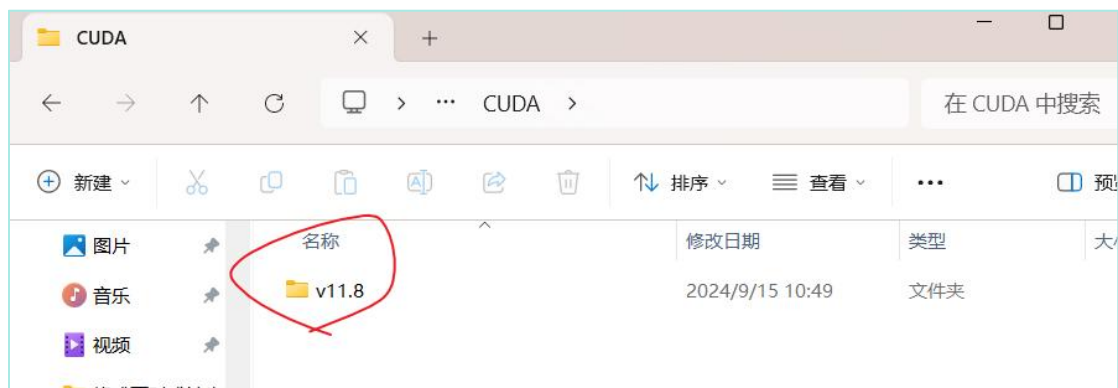


2.在环境变量中包含“cuda”的路径全部删除



3.在安装目录中删除原文件夹

找到你的安装目录，将这个文件夹删除。



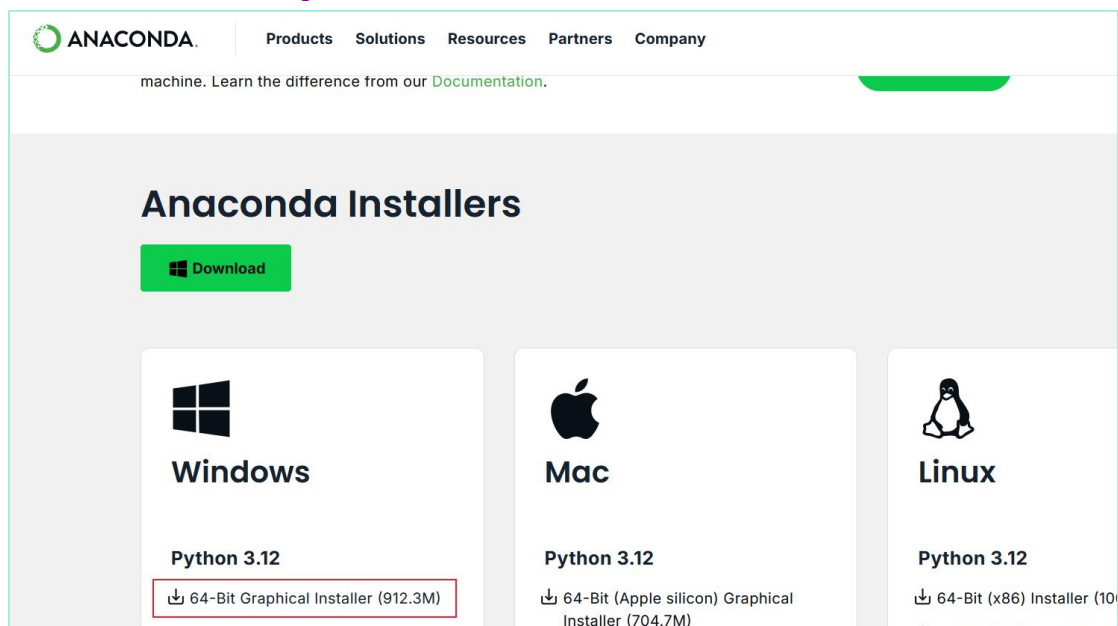
之后再重新安装就可以了。

1.7 anaconda 的安装（pycharm 的安装）

anaconda 提供了我们所需要的各种东西，能够快捷、方便的管理各方面的东西，注意是不需要单独下载 python 的。

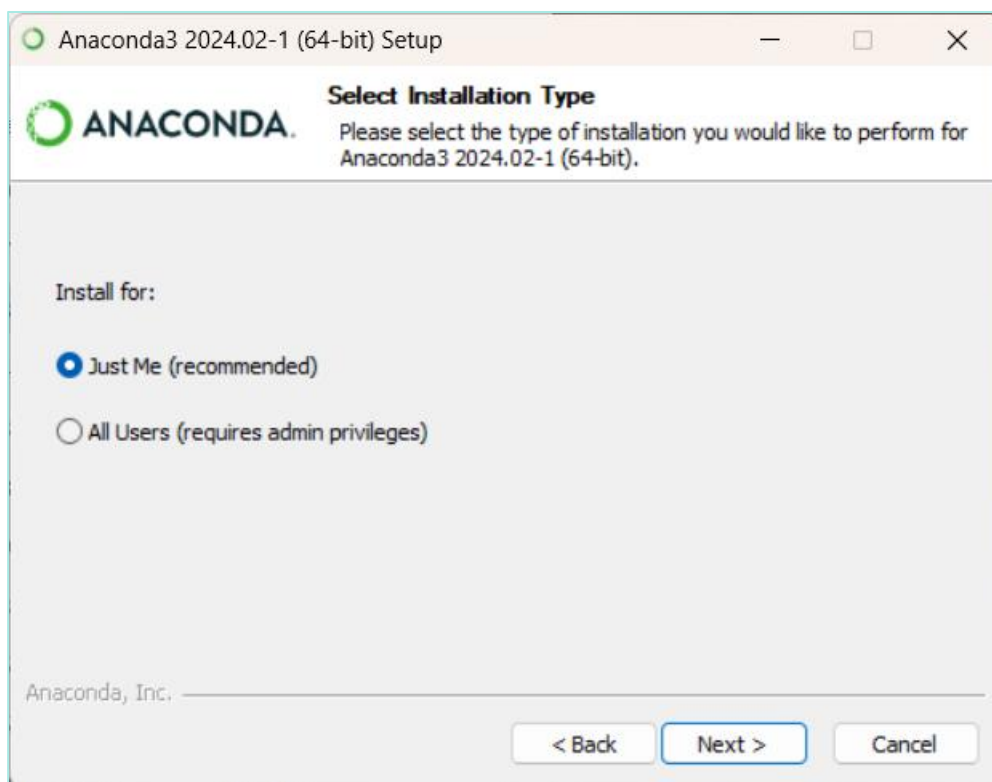
下载地址：

<https://www.anaconda.com/download/success>

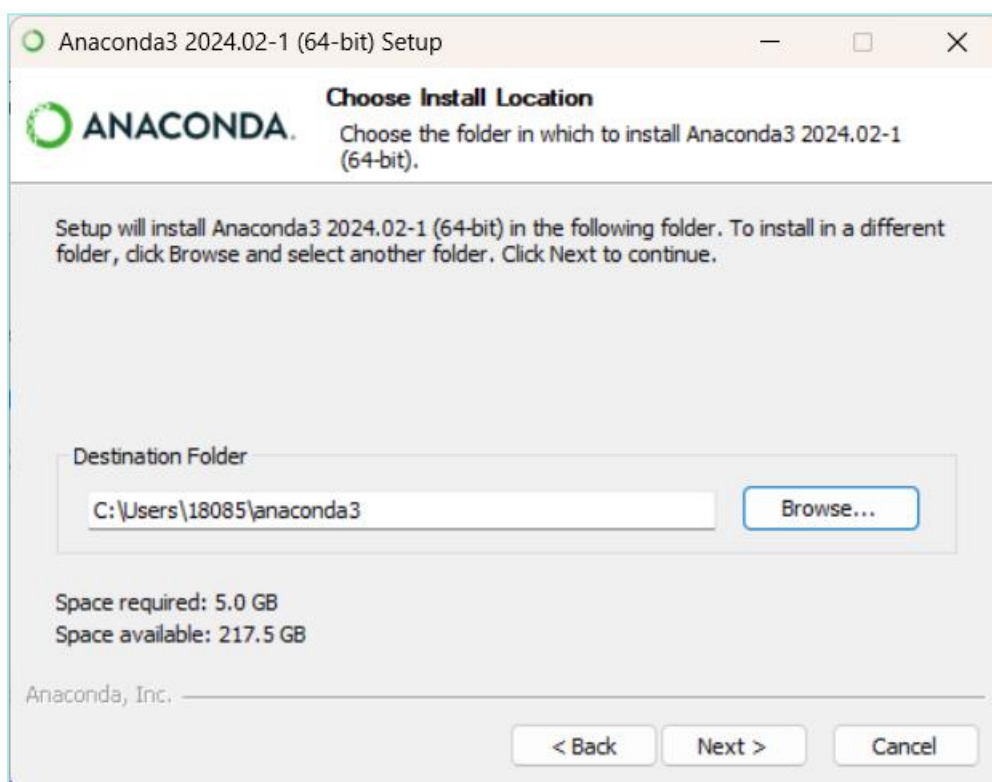


安装过程：

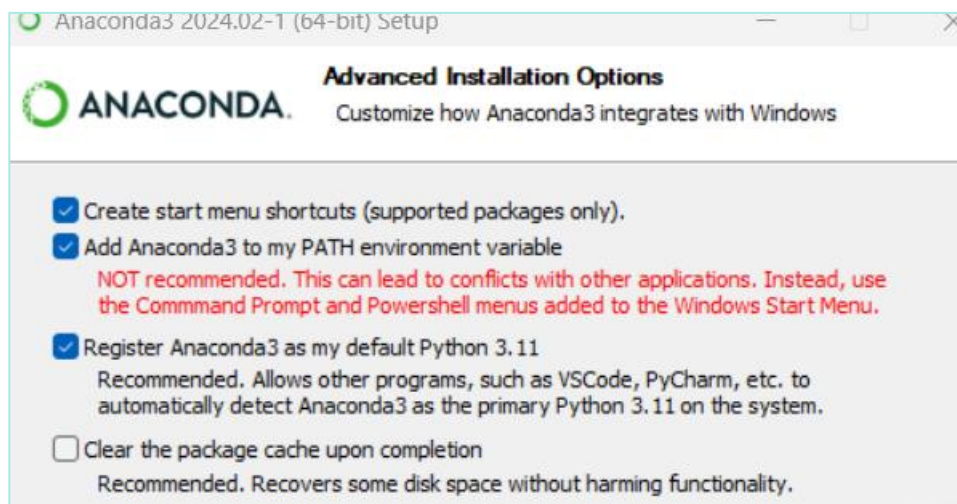
新手建议选择“just me”，选择另一个会比较麻烦



此时建议不要改路径（内存不够就去 D 盘）：



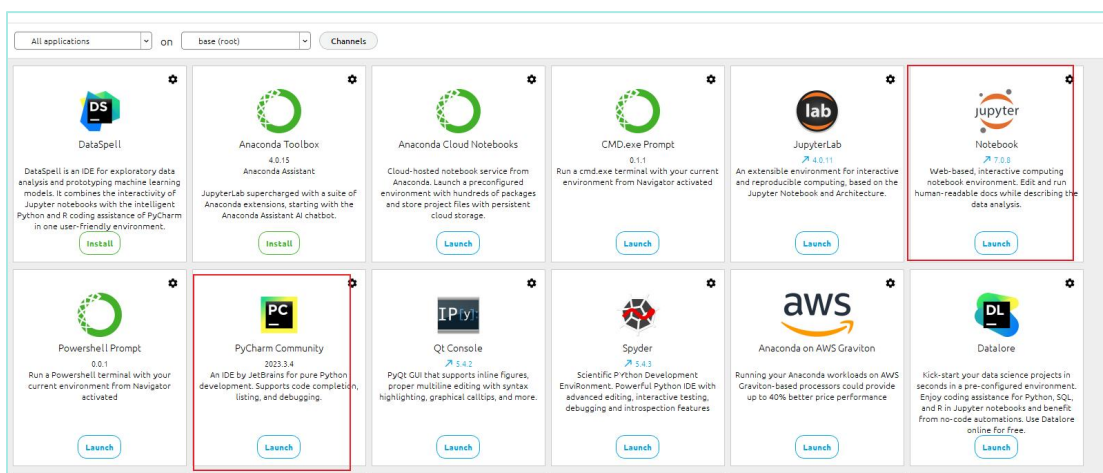
前三项打钩：



安装成功后，打开：



里面就有我们想要的各种东西，当然我们只需要 jupyter 和 pycharm（社区版），点击“launch”启动，就能自动帮我们下载了。



现在我们来验证 conda 是否可用，在控制台输入：

conda -V

```
C:\Users\18085>conda -V  
conda 24.1.2
```

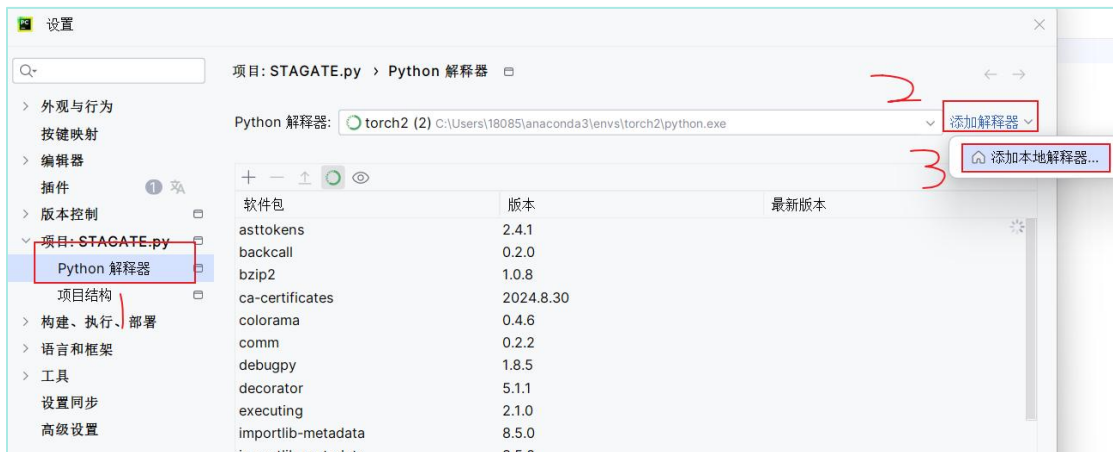
如果能正确显示就表明安装成功了。

如果 anaconda 没有显示 pycharm community 版本的，需要自行下载安装：

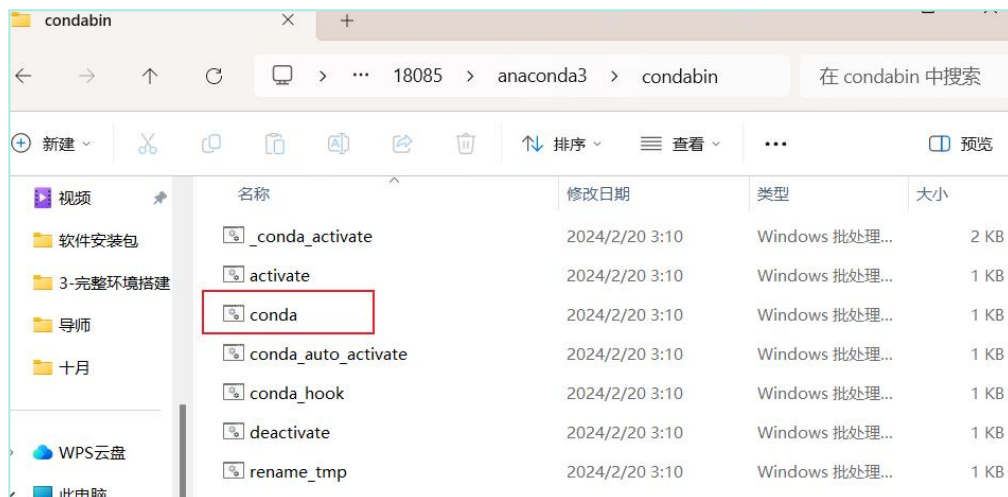
<https://www.jetbrains.com/pycharm/>

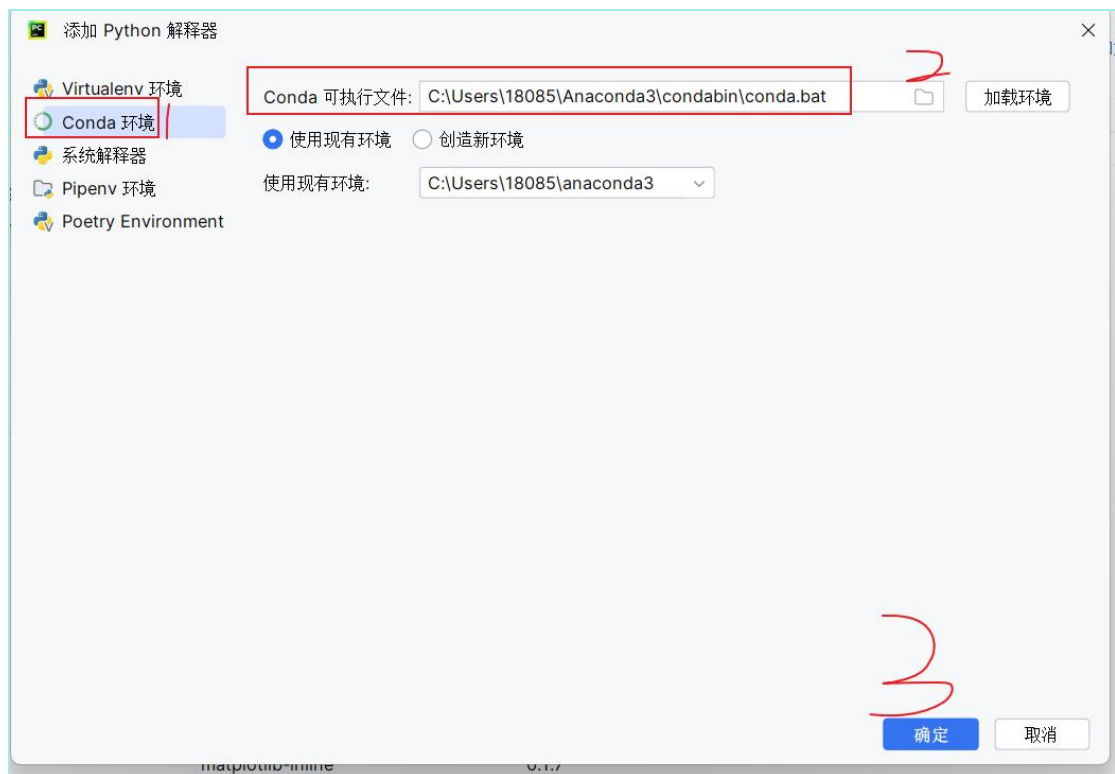


注意要选择社区版，然后安装完之后配置 anaconda 环境：



选择 conda 环境→找到你 python 安装的环境路径，找到类似下面的路径，点击确定即可。





1.8 anaconda 的路径设置（下载包会比较快）

如果不设置，并且不搭梯子，那样每次都要设置镜像源，比较麻烦。

参考：

https://blog.csdn.net/weixin_44914727/article/details/130513081

[以下以 pip 安装为例，conda 安装建议不要修改，一个就够了]

临时设置镜像源：

pip install some-package -i <https://pypi.tuna.tsinghua.edu.cn/simple>
some-package 表示你要装的包

永久设置镜像源（base 环境）：

pip config set global.index-url <https://pypi.tuna.tsinghua.edu.cn/simple>
那样以后 pip 都是从清华镜像安装了。

1.9 虚拟环境的创建

推荐不同项目使用不同的虚拟环境，后面吃亏了就懂了。

在控制台输入：

activate

```
C:\Users\18085>activate
C:\Users\18085>conda.bat activate
(base) C:\Users\18085>
```

即可进入 conda 环境

如果进不去，请将：

C:\Users\18085\anaconda3

C:\Users\18085\anaconda3\Scripts

加入环境变量。

请注意 base 是最基础的 conda 环境，请不要在里面随意安装各种包。

查看虚拟环境列表：

conda env list

```
(base) C:\Users\18085>conda env list
# conda environments:
#
base                * C:\Users\18085\anaconda3
torch2              C:\Users\18085\anaconda3\envs\torch2
```

创建自己的虚拟环境：

conda create -n mytest python=3.11

```
The following NEW packages will be INSTALLED:

bzip2                conda-forge/win-64::bzip2-1.0.8-h2466b09_7
ca-certificates      conda-forge/win-64::ca-certificates-2024.8.30-h56e8100_0
libexpat             conda-forge/win-64::libexpat-2.6.3-he0c23c2_0
libffi               conda-forge/win-64::libffi-3.4.2-h8ffe710_5
libsqlite            conda-forge/win-64::libsqlite-3.46.1-h2466b09_0
libzlib              conda-forge/win-64::libzlib-1.3.1-h2466b09_1
openssl              conda-forge/win-64::openssl-3.3.2-h2466b09_0
pip                  conda-forge/noarch::pip-24.2-pyh8b19718_1
python               conda-forge/win-64::python-3.11.10-hce54a09_0_cpython
setuptools           conda-forge/noarch::setuptools-73.0.1-pyhd8ed1ab_0
tk                   conda-forge/win-64::tk-8.6.13-h5226925_1
tzdata               conda-forge/noarch::tzdata-2024a-h8827d51_1
ucrt                 conda-forge/win-64::ucrt-10.0.22621.0-h57928b3_0
vc                   conda-forge/win-64::vc-14.3-h8a93ad2_21
vc14_runtime         conda-forge/win-64::vc14_runtime-14.40.33810-ha82c5b3_21
vs2015_runtime       conda-forge/win-64::vs2015_runtime-14.40.33810-h3bf8584_21
wheel                conda-forge/noarch::wheel-0.44.0-pyhd8ed1ab_0
xz                   conda-forge/win-64::xz-5.2.6-h8d14728_0

Proceed ([y]/n)? |
```

输入“y”按下回车即可。

再次输入：

conda env list

```
(base) C:\Users\18085>conda env list
# conda environments:
#
base                * C:\Users\18085\anaconda3
mytest              C:\Users\18085\anaconda3\envs\mytest
torch2              C:\Users\18085\anaconda3\envs\torch2
```

可以看到我们的虚拟环境创建成功了。进入虚拟环境：

activate mytest

```
(base) C:\Users\18085>activate mytest
(mytest) C:\Users\18085>
```

然后就可以在 mytest 这个虚拟环境进行相关操作了。

各个虚拟环境之间相互独立，这个虚拟环境安装的包不会在另一个虚拟环境中显示出来。更多虚拟环境的操作在这里就不赘述了，需要的时候自行查阅。

1.9 虚拟环境（补充）

克隆虚拟环境，相当于创建一个虚拟环境一模一样的副本，eg: 将 torch3 里面所有的包克隆到新环境 torchSTalign:

```
conda create --name torchSTalign --clone torch3
```

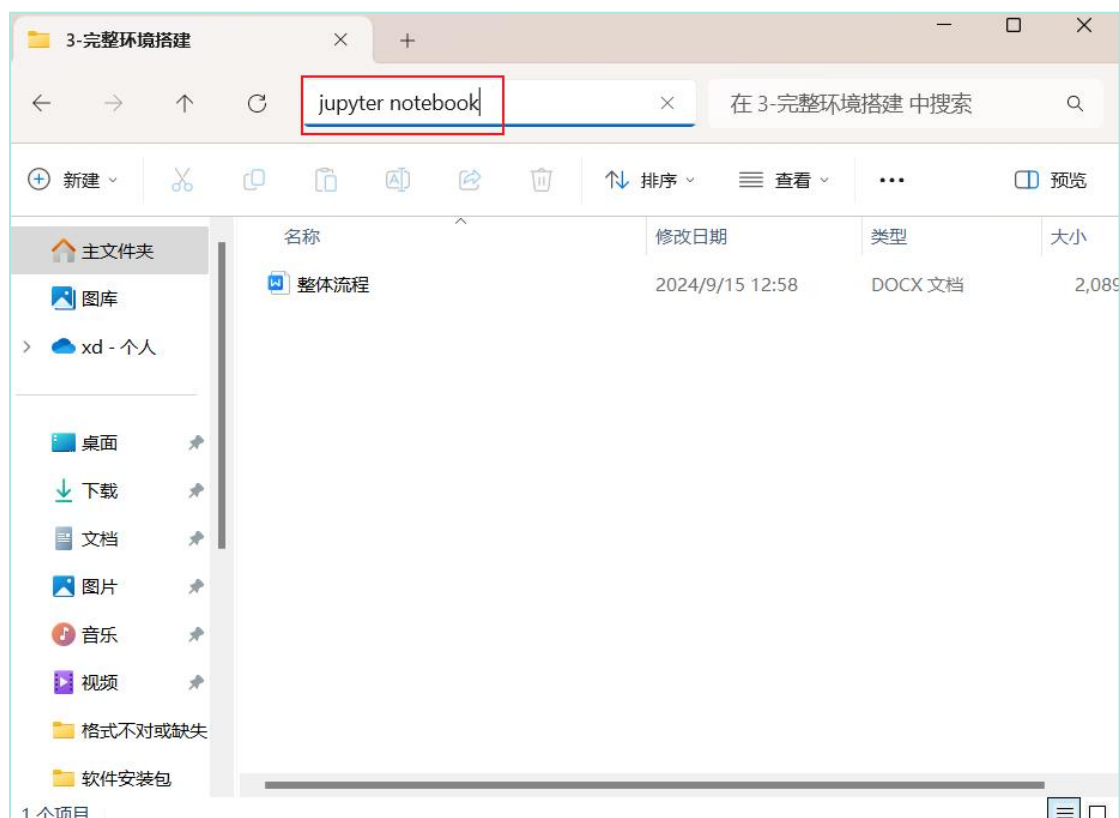
```
(base) C:\Users\18085>activate torchSTalign
(torchSTalign) C:\Users\18085>
```

删除虚拟环境:

```
conda remove -n SDMBench --all
```

1.10 虚拟环境的使用（jupyter）

在 jupyter notebook 中配置虚拟环境，最简单的方式打开 jupyter notebook，在任意路径输入 jupyter notebook，然后按回车即可：



进入会使用你的默认浏览器进入，注意，打开的同时会打开与之对应的控制台，

```
C:\Python_install\Python311\5 x + v
b
[I 2024-09-15 12:59:49.086 LabApp] JupyterLab application directory is C:\Users\18085\anaconda3\share\jupyter\lab
[I 2024-09-15 12:59:49.086 LabApp] Extension Manager is 'pypi'.
[I 2024-09-15 12:59:49.087 ServerApp] jupyterlab | extension was successfully loaded.
[I 2024-09-15 12:59:49.092 ServerApp] notebook | extension was successfully loaded.
[I 2024-09-15 12:59:49.093 ServerApp] panel.io.jupyter_server_extension | extension was successfully loaded.
[I 2024-09-15 12:59:49.093 ServerApp] Serving notebooks from local directory: C:\Users\18085\Desktop\b站\pytorch学习\3-完整环境搭建
[I 2024-09-15 12:59:49.094 ServerApp] Jupyter Server 2.10.0 is running at:
[I 2024-09-15 12:59:49.094 ServerApp] http://localhost:8888/tree?token=b19c68497187b1b405706abeb0573662f669ee7650749ec1
[I 2024-09-15 12:59:49.094 ServerApp] http://127.0.0.1:8888/tree?token=b19c68497187b1b405706abeb0573662f669ee7650749ec1
[I 2024-09-15 12:59:49.094 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2024-09-15 12:59:49.190 ServerApp]

To access the server, open this file in a browser:
file:///C:/Users/18085/AppData/Roaming/jupyter/runtime/jpserver-13784-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=b19c68497187b1b405706abeb0573662f669ee7650749ec1
http://127.0.0.1:8888/tree?token=b19c68497187b1b405706abeb0573662f669ee7650749ec1
[I 2024-09-15 12:59:49.319 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
0.00s - Debugger warning: It seems that frozen modules are being used, which may
0.00s - make the debugger miss breakpoints. Please pass -Xfrozen_modules=off
0.00s - to python to disable frozen modules.
0.00s - Note: Debugging will proceed. Set PYDEVD_DISABLE_FILE_VALIDATION=1 to disable this validation.
```

请不要关闭！！！！关闭了jupyter就无响应了。

（如果jupyter打不开，报错如下：）

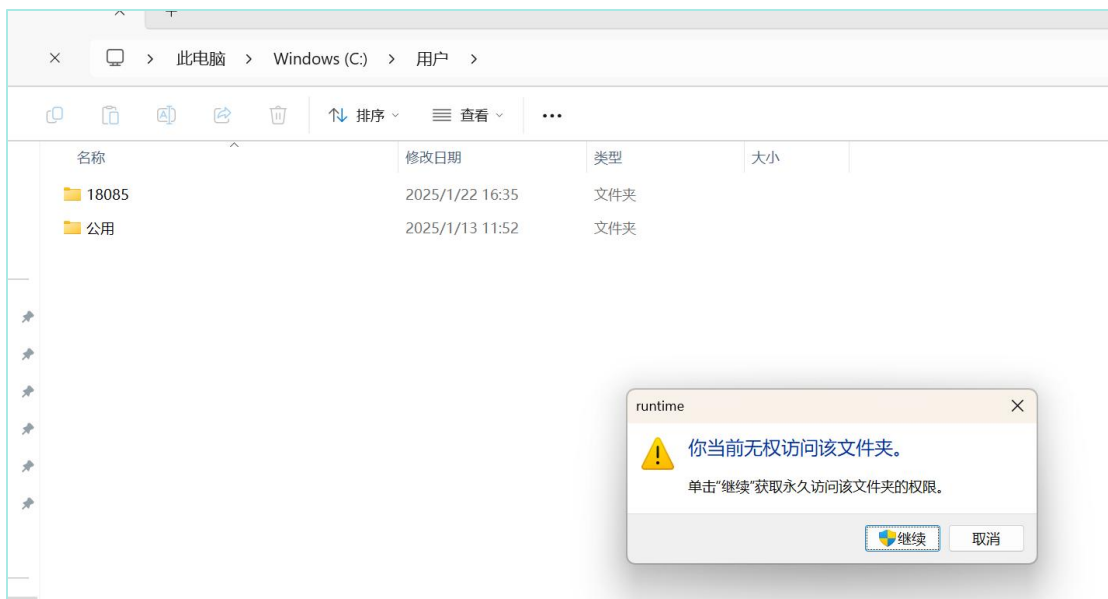
对该文件的访问请求被拒绝

无法读取 /C:/Users/18085/AppData/Roaming/jupyter/runtime/jpserver-20284-open.html 上的该文件。

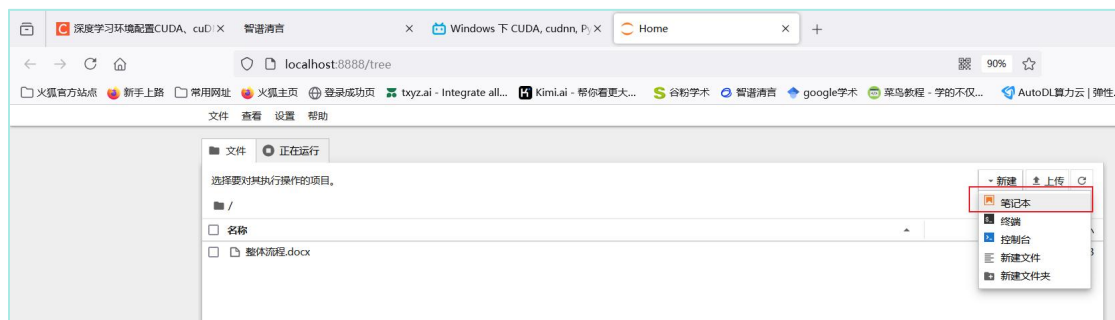
- 文件可能已被删除、移动，或者因文件权限问题被拒绝访问。

重试

切换目录至：C:/Users/18085/AppData/Roaming/jupyter/runtime 获得权限



新建选择:

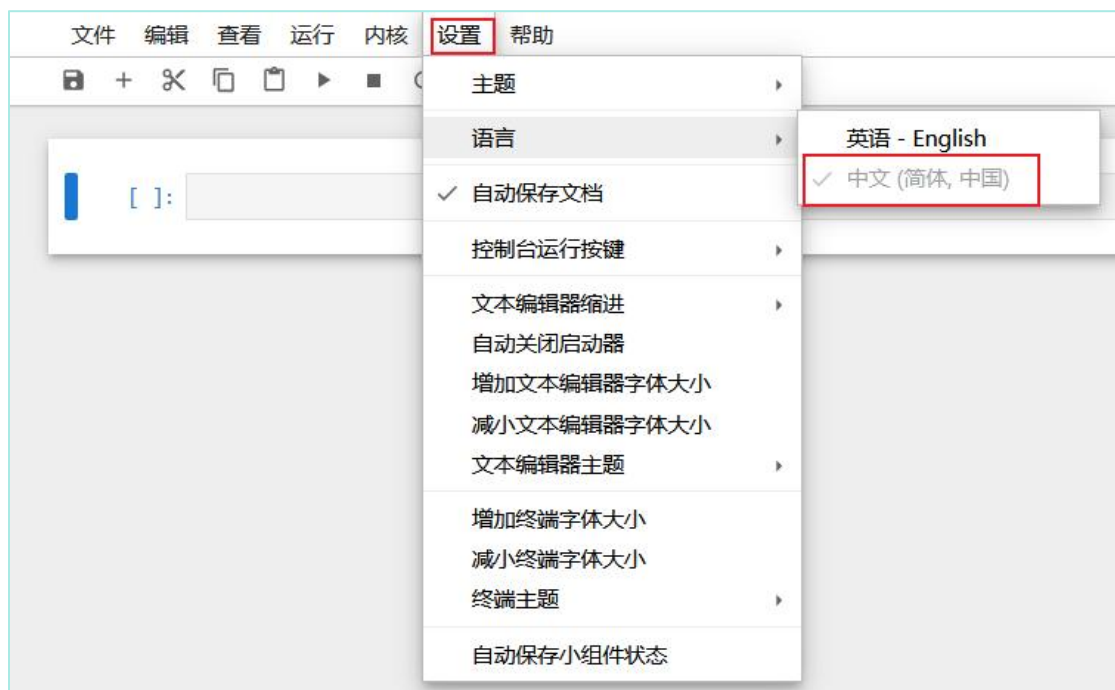


如果你想配置 jupyter 为中文，请在控制台输入：

`pip install jupyterlab-language-pack-zh-CN`

```
(base) C:\Users\18085>pip install jupyterlab-language-pack-zh-CN
```

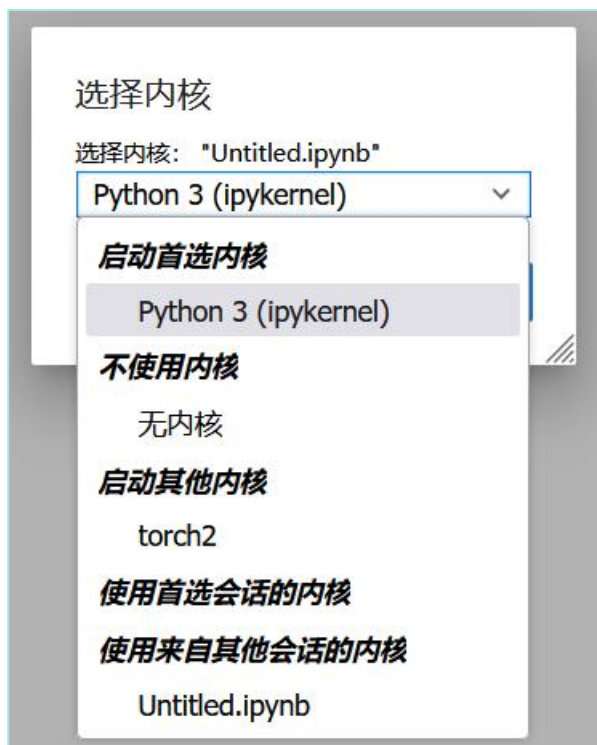
然后找到设置选择中文就好了：



这样打开的 jupyter 就是中文啦：



配置虚拟环境到 jupyter 内核，此时我们可以发现，我们刚刚创建的虚拟环境用不了：

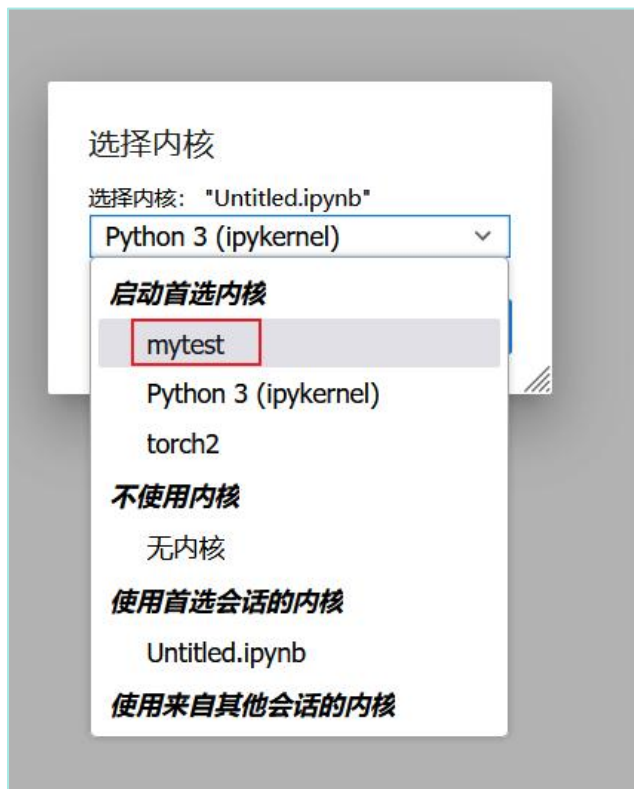


在控制台（切记是当前所选的虚拟环境）输入：

```
python -m ipykernel install --user --name=mytest
```

```
(base) C:\Users\18085>python -m ipykernel install --user --name=mytest
0.00s - Debugger warning: It seems that frozen modules are being used, which may
0.00s - make the debugger miss breakpoints. Please pass -Xfrozen_modules=off
0.00s - to python to disable frozen modules.
0.00s - Note: Debugging will proceed. Set PYDEVD_DISABLE_FILE_VALIDATION=1 to disable this validation.
Installed kernelspec mytest in C:\Users\18085\AppData\Roaming\jupyter\kernels\mytest
```

刷新 jupyter 就可以选择啦



如果上述配置有报错显示没有 ipykernel 模块, 请先在那个虚拟环境中(不是 base 环境) 安装 ipykernel:

```
conda install ipykernel
```

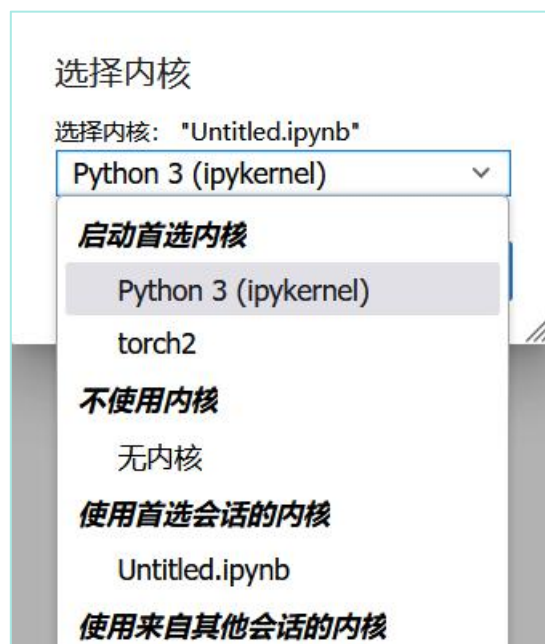
然后再添加内核就好啦。

移除内核:

```
jupyter kernelspec remove mytest
```

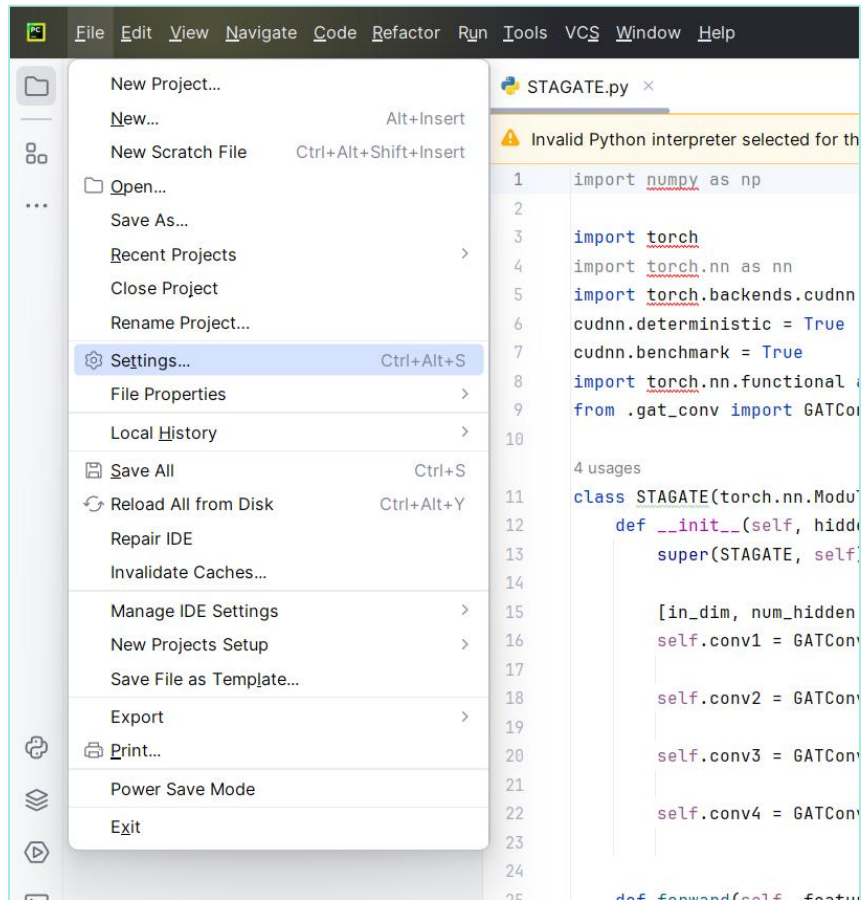
```
(base) C:\Users\18085>jupyter kernelspec remove mytest
Kernel specs to remove:
  mytest C:\Users\18085\AppData\Roaming\jupyter\kernels\mytest
Remove 1 kernel specs [y/N]: y
Removed C:\Users\18085\AppData\Roaming\jupyter\kernels\mytest
```

这样 jupyter 里面就没有这个内核了。

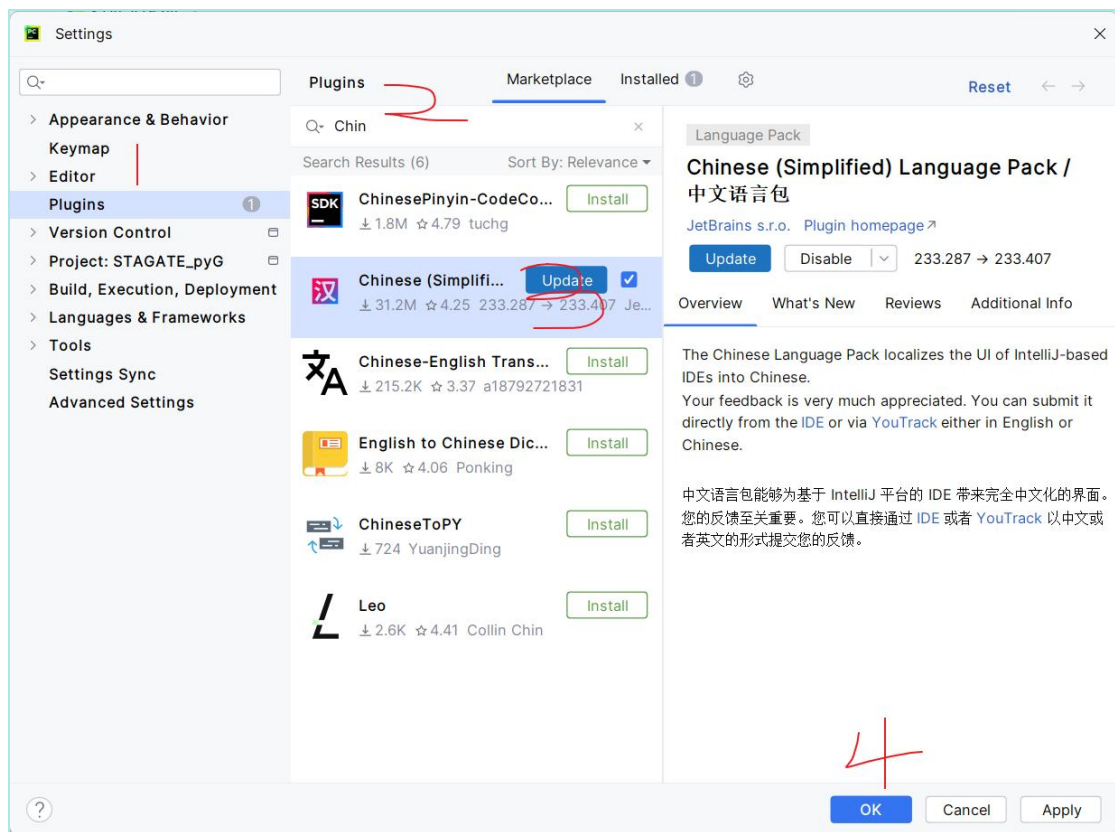


1.11 虚拟环境的使用（pycharm）

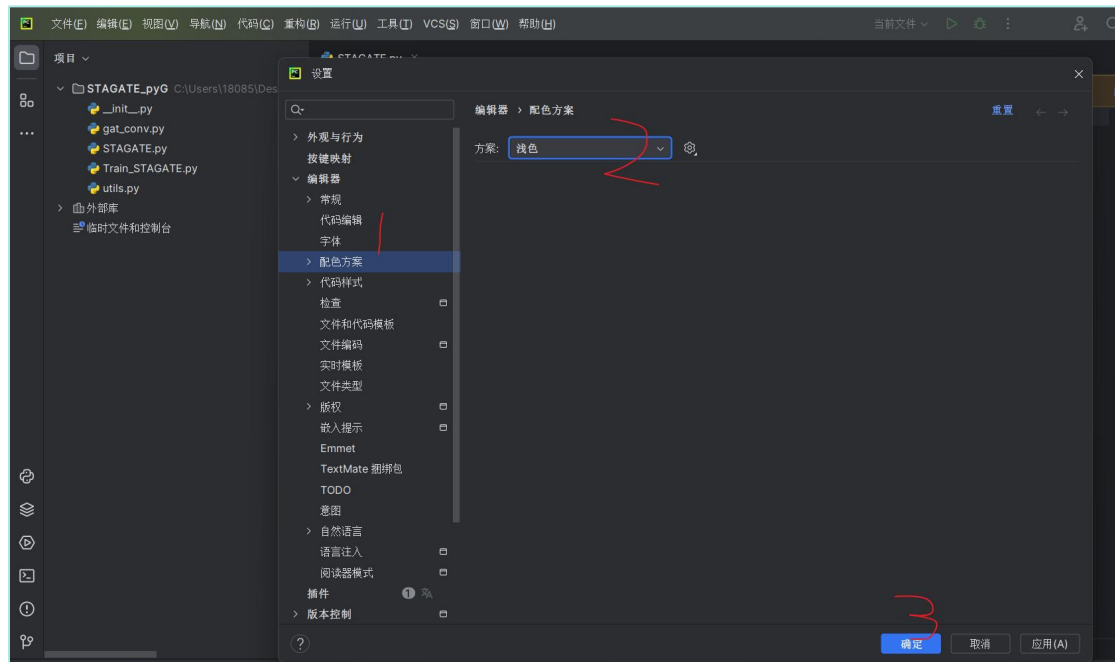
pycharm 汉化，（如果有需要的话）



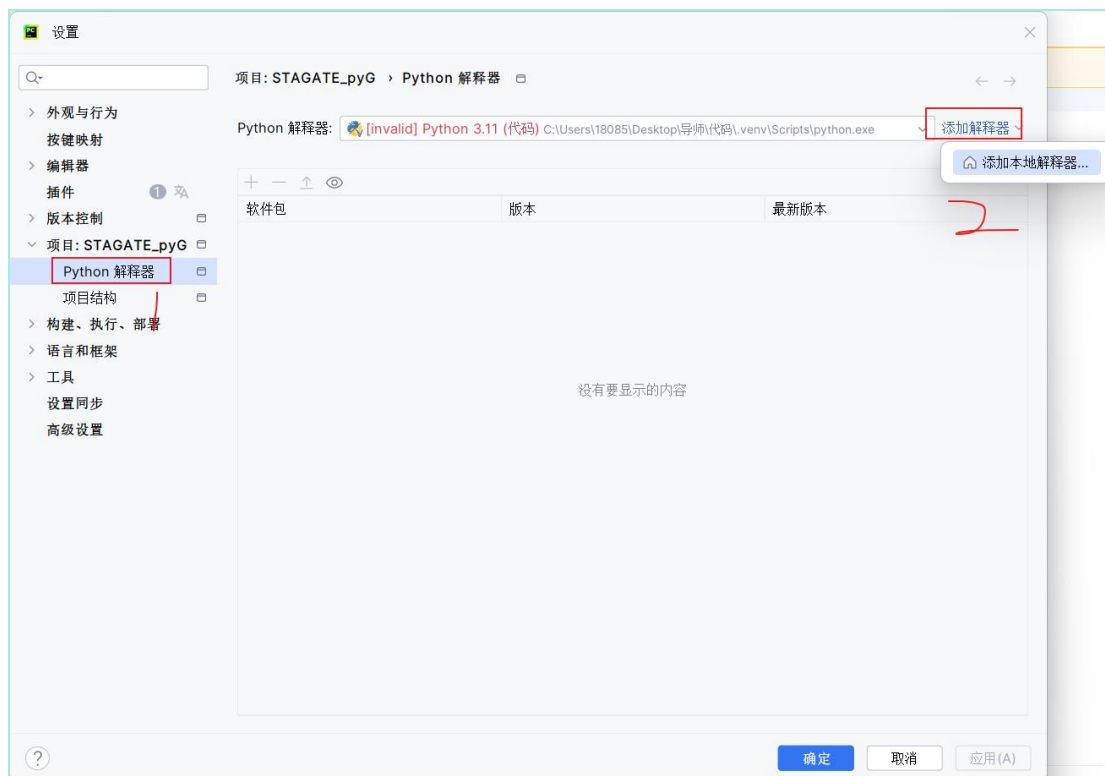
随后按照步骤：



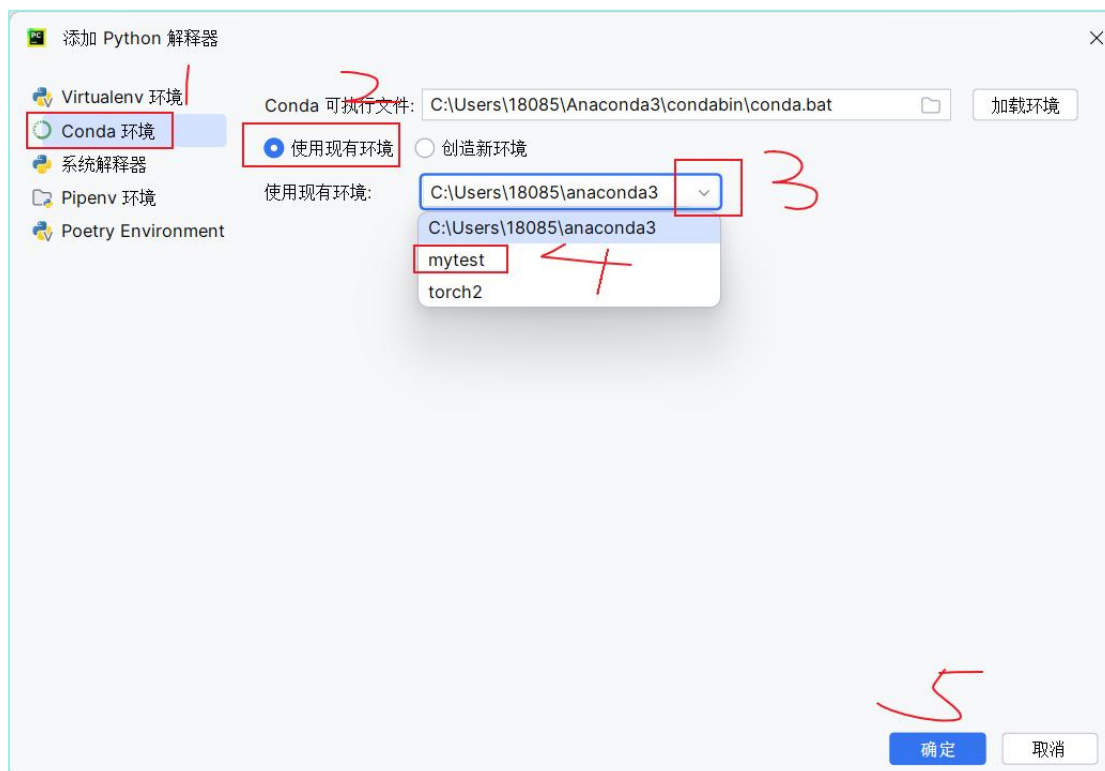
pycharm 变成白底配色:



接下来配置虚拟环境，在设置中找到 python 解释器:



按照以下步骤选择就好啦，那么这个项目所用的解释器就是虚拟环境 mytest 中的 python 了。



1.12 安装 pytorch 框架

进入 pytorch 官网获取安装指令：

<https://pytorch.org/>

下滑找到：

NOTE: Latest PyTorch requires Python 3.8 or later.

PyTorch Build	Stable (2.4.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	CUDA 12.4	ROCm 6.1
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

Previous versions of PyTorch > ←

圆圈处就是你的安装命令，箭头表示更早的 pytorch 版本。复制到控制台，进入你想要安装的虚拟环境，安装即可：

```
(torch2) C:\Users\18085>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

验证是否安装成功：

首先输入

python

```
(torch2) C:\Users\18085>python
Python 3.8.19 | packaged by conda-forge | (default, Mar 20 2024, 12:38:07) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

再输入下述指令就好了：

import torch

torch.cuda.is_available()

torch.version.cuda

```
>>> import torch
>>> torch.cuda.is_available()
True
>>> torch.version.cuda
'11.8'
>>> |
```

能正常显示就表示没问题了。

当然将上述指令也可以复制到 jupyter 和 pycharm 中，如果返回一样的结果，就说明虚拟环境已经配置 pytorch 成功。

2 各种代码问题总结

2.1 python 自己包的配置命令

```
cd STAGATE_pyG-main
python setup.py build
python setup.py install
```

注意加上: zip_safe = False,

2.2 Windows 的 pyG 安装方式

```
pip install --no-index torch-scatter -f https://pytorch-geometric.com/whl/torch-1.9.0+cu111.html
pip install --no-index torch-sparse -f https://pytorch-geometric.com/whl/torch-1.9.0+cu111.html
pip install --no-index torch-cluster -f https://pytorch-geometric.com/whl/torch-1.9.0+cu111.html
pip install --no-index torch-spline-conv -f https://pytorch-geometric.com/whl/torch-1.9.0+cu111.html
pip install torch-geometric
```

linux 直接 pip 装就好了, 系统会帮你解决。

2.3 R 相关配置

linux 安装 R 命令:

```
conda create -n r_env -c conda-forge r=4.2.0
```

Windows 配置环境:

```
os.environ['R_USER'] =
r'C:\Users\18085\anaconda3\envs\torch3\Lib\site-packages\rpy2'
os.environ['R_HOME'] = 'C:\Program Files\R\R-4.3.3'
```

2.4 项目路径设置

```
# 查看当前工作目录
import os
print(os.getcwd())
# 添加
import sys
sys.path.append(r'xxx')
# 查看所有目录路径
import sys
for i in range(0, len(sys.path)):
    print(sys.path[i])
```

```
# 删除
sys.path.remove(r'xxx')

# 查看 python 解释器的命令
import sys
print(sys.executable)

which python

# 查看 jupyter 内核
!jupyter kernelspec list
```

2.5 linux 命令总结

```
nvidia-smi
ls -la 【显示隐藏文件】
cat setup.py 【查看文件】
cd - 【返回上一层路径并返回】
cd ..
cd / cd~ 回到开始路径
【快速进入相关路径】
find . -name setup.py
cd ./some_directory
切换快捷命令: cd "$(dirname "$(find . -name setup.py | head -n 1)")"
du -sh * 【查看用户内存使用情况】
df -h 【查看硬盘内存情况】
ctrl + u 【清空命令行】
rm -rf /home/user/test 【删除文件夹】
```

修改配置文件

```
vim ~/.bashrc
输入命令后
保存并退出编辑器（在 vim 中按 Esc，然后输入 :wq 回车）
source ~/.bashrc 配置文件让文件生效
```


2.5.1 文件与目录操作（7个）

这是最基础也最常用的指令，用于管理文件和目录的创建、查看、删除等。

- **ls**: 列出目录内容
 - 常用选项:
 - **-l**: 以详细列表形式显示 (权限、所有者、大小、修改时间等)
 - **-a**: 显示所有文件 (包括隐藏文件, 以.开头)
 - **-h**: 与**-l**配合, 以人类易读的单位显示文件大小 (如 KB、MB)
 - 示例: **ls -lha** 显示当前目录所有文件的详细信息, 包括隐藏文件, 大小以易读单位呈现。

ps: ll 等价于 ls -l

- **cd**: 切换目录
 - 示例:
 - **cd /home/user**: 切换到/home/user目录
 - **cd ..**: 切换到上一级目录
 - **cd ~**: 切换到当前用户的家目录
 - **cd -**: 切换到上一次所在的目录

- **pwd**: 显示当前工作目录的绝对路径
 - 示例: 执行**pwd**后, 可能输出/home/user/documents。

- **mkdir**: 创建目录
 - 常用选项: **-p** 递归创建目录 (若父目录不存在则自动创建)
 - 示例: **mkdir -p dir1/dir2** 同时创建dir1和其下的dir2目录。

- **rm**: 删除文件或目录
 - 常用选项:
 - **-f**: 强制删除, 不提示
 - **-r**: 递归删除目录及其内容 (删除目录必须用此选项)
 - 示例: **rm -rf old_dir** 强制删除old_dir目录及其所有内容 (**谨慎使用, 删除后无法恢复**)。

- **cp**: 复制文件或目录
 - 常用选项:
 - **-r**: 递归复制目录（复制目录必须用此选项）
 - **-p**: 保留文件的权限、所有者等属性
 - 示例: `cp -rp source_dir target_dir` 将source_dir目录及内容复制到target_dir, 并保留属性。
- **mv**: 移动文件或目录, 也可用于重命名
 - 示例:
 - `mv file1 /home/user`: 将file1移动到/home/user目录
 - `mv old_name new_name`: 将old_name重命名为new_name

创建 txt 文档

1. 使用touch命令 (推荐) :

```
bash ^
touch new.txt
```

这会在当前目录下创建一个空的new.txt文件, 如果文件已存在, 会更新其修改时间而不改变内容。

2. 使用文本编辑器创建 (同时可以直接编辑内容) :

- 用nano编辑器:

```
bash ^
nano new.txt
```

编辑完成后按Ctrl+O保存, Ctrl+X退出。

- 用vim编辑器:

```
bash ^
vim new.txt
```

进入后按i键开始编辑, 完成后按Esc, 再输入:wq保存退出。

3. 重定向空输出创建空文件:

```
bash ^
> new.txt
```

这会创建一个空文件, 如果文件已存在会清空其内容。

4. 使用echo命令创建并写入内容:

```
bash ^
echo "这是文件内容" > new.txt
```

这会创建new.txt并写入指定内容, 如果文件已存在会覆盖原有内容。

2.5.2 文件内容查看与处理（5 个）

用于查看文件内容、搜索文本、统计文件信息等。

- **cat**: 查看文件内容（一次性显示全部内容，适合短文件）
 - 示例: `cat test.txt` 显示test.txt的全部内容。
- **more/less**: 分页查看文件内容（适合长文件）
 - **more**: 按页显示，支持空格键翻页，q退出
 - **less**: 功能更强大，支持上下键滚动、搜索（/关键词向下搜，?关键词向上搜），q退出
 - 示例: `less long_file.txt` 分页查看long_file.txt，可灵活滚动和搜索。
- **head/tail**: 查看文件开头或结尾内容
 - `head -n 5 file.txt`: 显示file.txt的前 5 行（默认前 10 行）
 - `tail -n 5 file.txt`: 显示file.txt的后 5 行（默认后 10 行）
 - `tail -f log.txt`: 实时跟踪log.txt的新增内容（常用于查看日志）
- **grep**: 在文件中搜索匹配的文本
 - 常用选项:
 - `-i`: 忽略大小写
 - `-n`: 显示匹配行的行号
 - `-r`: 递归搜索目录下的所有文件
 - 示例: `grep -inr "error" /var/log` 在/var/log目录下所有文件中，忽略大小写搜索包含“error”的行，并显示行号。
- **wc**: 统计文件的行数、单词数、字节数
 - 选项: `-l` (行数)、`-w` (单词数)、`-c` (字节数)
 - 示例: `wc -l file.txt` 统计file.txt的行数。

2.5.3 系统管理与状态查看（7 个）

用于查看系统资源、进程状态、系统信息等。

- **top/htop**: 查看系统进程和资源占用（htop需单独安装，界面更友好）
 - top中按q退出，按P按 CPU 占用排序，按M按内存占用排序。
- **ps**: 查看进程状态
 - 常用选项:
 - `ps aux`: 显示系统中所有进程的详细信息（a所有用户进程，u显示用户，x包括非终端进程）
 - `ps -ef`: 以全格式显示所有进程
 - 示例: `ps aux | grep python` 查看所有包含“python”的进程。

- **kill**: 终止进程
 - 格式: `kill [信号] 进程ID` (默认信号为 15, 请求进程退出; 信号 9 为强制终止)
 - 示例: `kill 1234` 终止 ID 为 1234 的进程; `kill -9 1234` 强制终止该进程。
- **df**: 查看磁盘空间使用情况
 - 常用选项: `-h` 以易读单位显示
 - 示例: `df -h` 显示各分区的总容量、已用、可用空间及挂载点。
- **free**: 查看内存使用情况
 - 常用选项: `-h` 以易读单位显示
 - 示例: `free -h` 显示总内存、已用、空闲、缓存等信息。

- **uname**: 查看系统信息
 - 常用选项: `-a` 显示所有系统信息 (内核版本、主机名、硬件架构等)
 - 示例: `uname -a` 输出完整的系统信息。
- **uptime**: 查看系统运行时间及负载
 - 示例: 输出可能为 `12:34:56 up 2 days, 3:45, 2 users, load average: 0.12, 0.34, 0.56`, 表示系统已运行 2 天 3 小时 45 分, 平均负载分别为 1 分钟、5 分钟、15 分钟的数值。

2.5.4 用户与权限管理 (5 个)

用于管理用户、组及文件权限。

- **useradd/userdel**: 创建 / 删除用户 (需管理员权限, 通常用 `sudo`)
 - 示例: `sudo useradd -m newuser` 创建 `newuser` 并自动创建家目录 (`-m`); `sudo userdel -r newuser` 删除 `newuser` 并同时删除家目录 (`-r`)。
- **passwd**: 修改用户密码
 - 示例: `passwd` 修改当前用户密码; `sudo passwd newuser` 管理员修改 `newuser` 的密码。
- **groupadd/groupdel**: 创建 / 删除用户组
 - 示例: `sudo groupadd dev` 创建 `dev` 组; `sudo groupdel dev` 删除 `dev` 组。
- **chmod**: 修改文件或目录的权限
 - 权限表示: `r` (读, 4)、`w` (写, 2)、`x` (执行, 1), 分别对应所有者 (`u`)、所属组 (`g`)、其他用户 (`o`)
 - 示例:
 - `chmod 755 file.sh`: 所有者有读 / 写 / 执行权限 ($7=4+2+1$), 所属组和其他用户有读 / 执行权限 ($5=4+1$)
 - `chmod u+x file.sh`: 给所有者添加执行权限
- **chown**: 修改文件或目录的所有者和所属组
 - 示例: `sudo chown user:group file.txt` 将 `file.txt` 的所有者改为 `user`, 所属组改为 `group`。

2.5.5 网络操作（4 个）

用于查看网络状态、连接、下载文件等。

- **ifconfig/ip**: 查看或配置网络接口 (**ifconfig**逐渐被**ip**替代)
 - 示例: `ip addr` 显示所有网络接口的 IP 地址等信息。
- **ping**: 测试与目标主机的网络连通性
 - 示例: `ping www.baidu.com` 向百度服务器发送 ICMP 请求, 测试是否可达。
- **netstat/ss**: 查看网络连接状态 (**ss**更高效, 推荐使用)
 - 示例: `ss -tuln` 显示所有监听的 TCP (t) 和 UDP (u) 端口, 以数字形式 (n) 显示, 不解析域名。
- **wget/curl**: 下载网络文件
 - 示例: `wget https://example.com/file.zip` 下载文件; `curl -O https://example.com/file.zip` 同样下载文件 (-O保存为原文件名)。

2.5.6 压缩与解压（3 个）

用于处理压缩文件（如 .tar、.zip、.gz 等）。

- **tar**: 处理 .tar、.tar.gz、.tar.bz2 等格式
 - 常用选项:
 - `-c`: 创建压缩文件
 - `-x`: 解压文件
 - `-v`: 显示过程
 - `-f`: 指定文件名
 - `-z`: 用 gzip 压缩 / 解压 (.tar.gz)
 - `-j`: 用 bzip2 压缩 / 解压 (.tar.bz2)
 - 示例:
 - `tar -zcvf archive.tar.gz dir1` 将 dir1 压缩为 archive.tar.gz
 - `tar -zxvf archive.tar.gz` 解压 archive.tar.gz 到当前目录
- **zip/unzip**: 处理 .zip 格式
 - 示例: `zip -r archive.zip dir1` 压缩 dir1 为 archive.zip; `unzip archive.zip` 解压 archive.zip。

以上是 Linux 中最常用的一批指令, 掌握它们能满足日常的系统操作需求。实际使用中, 可通过 `man` 指令名 (如 `man ls`) 查看指令的完整手册, 了解更多选项和用法。

2.5.7 周总提供

Linux 使用什么命令查看某个端口被占用/如何在 Linux 系统中查看 TCP 状态?

`netstat` 命令(networkstatistics), `netstat -napt`

如何结束进程名为 aaa 的进程?

首先可以使用 `pgrep` 命令查找它的 PID, `pgrep aaa` 会返回所有名为 aaa 的进程的 PID; 然后使用 `kill` 命令结束进程, `kill` 命令会向指定进程发送信号, 默认

发送的是 SIGTERM 信号，该信号会请求进程正常终止，进程在收到信号后会进行一些清理工作然后终止，kill1234 可以结束 PID 为 1234 的进程;如果进程对 SIGTERM 信号无响应，还可以使用 kill-9 PID 来强制结束进程-9 表示发送 SIGKILL 信号，会强制结束进程，但这种方式可能会导致进程没有机会进行清理操作，有可能丢失数据。

常用Linux命令

- grep: 筛选符合某种文本的内容，通常搭配管道"|"使用。
- ps(process status): 查看当前的进程信息。
- top: 实时显示系统中各个进程的资源占用情况。
- kill/pkill: 用于终止进程，-9 是强制终止，-15 是默认信号，请求进程正常。kill 通过 pid 进行终止，pkill 是可以通过进程名进行终止。
- pgrep: 根据进程名或其他属性查找进程的 PID。

chmod +x 可以给文件添加执行权限

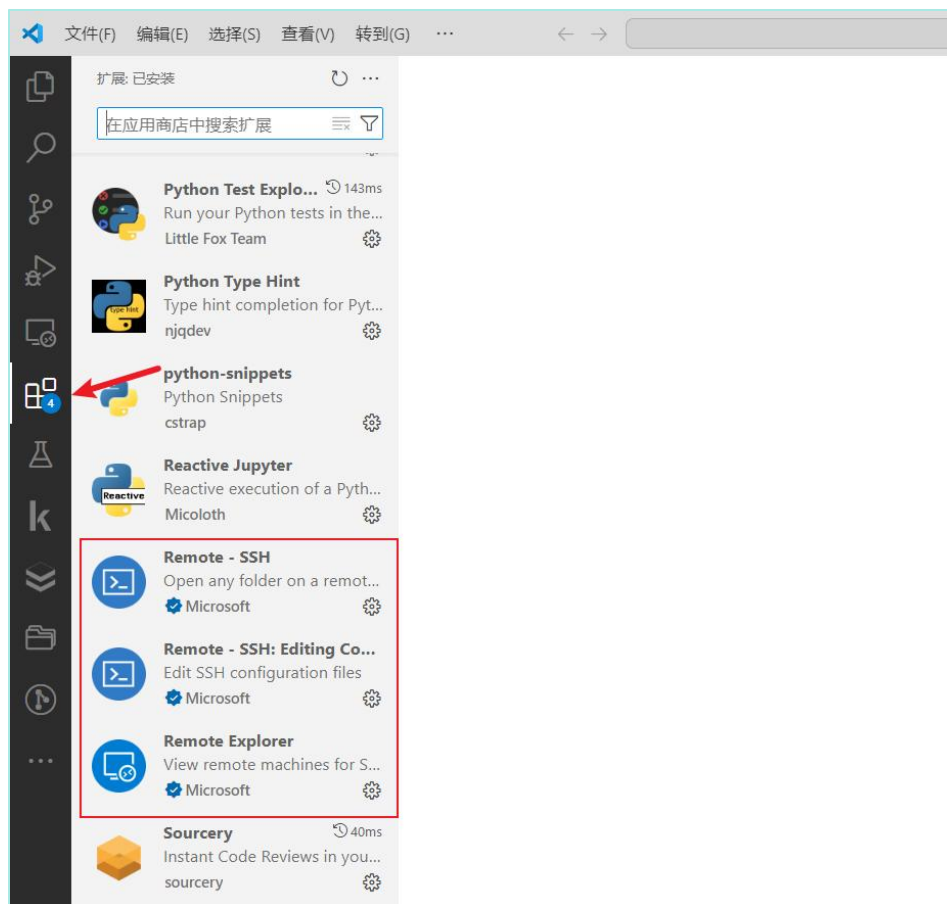
你需要通过给我提需求(布置任务的方式)，让我熟悉 linux 命令中的"`|`"、"`>`"、"`>>`"、"`2>`"、"`&>`"、"`2>&1`"以及"`tee`"

2.6 Python→requirements 文件

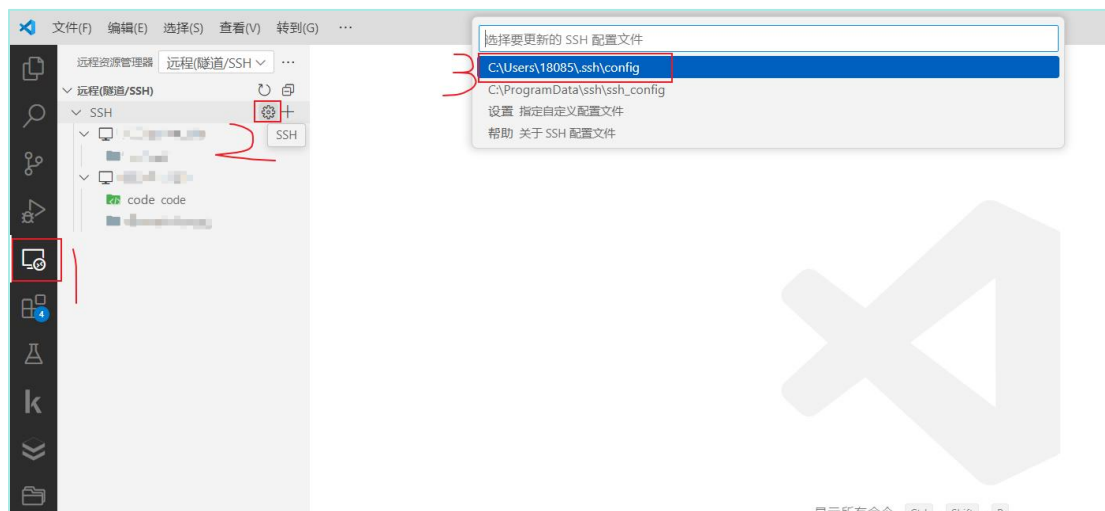
```
pip freeze > C:\Users\18085\Desktop\requirements.txt
pip install -r requirements.txt
```

2.7 VS 连接远程服务器

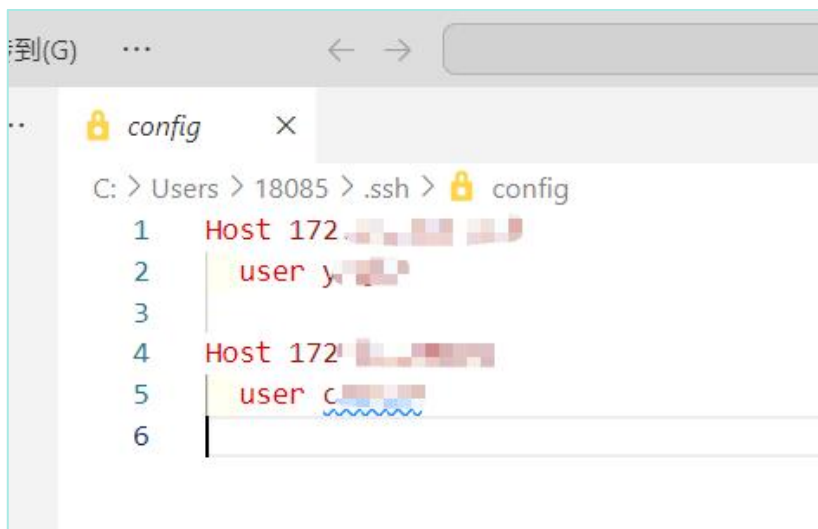
首先假设你已经下好 VS code 了，安装以下拓展（可以把和 Remote 的都装了哈哈哈，开始我虚拟环境一直过不来，原来是要装第二个拓展。）：



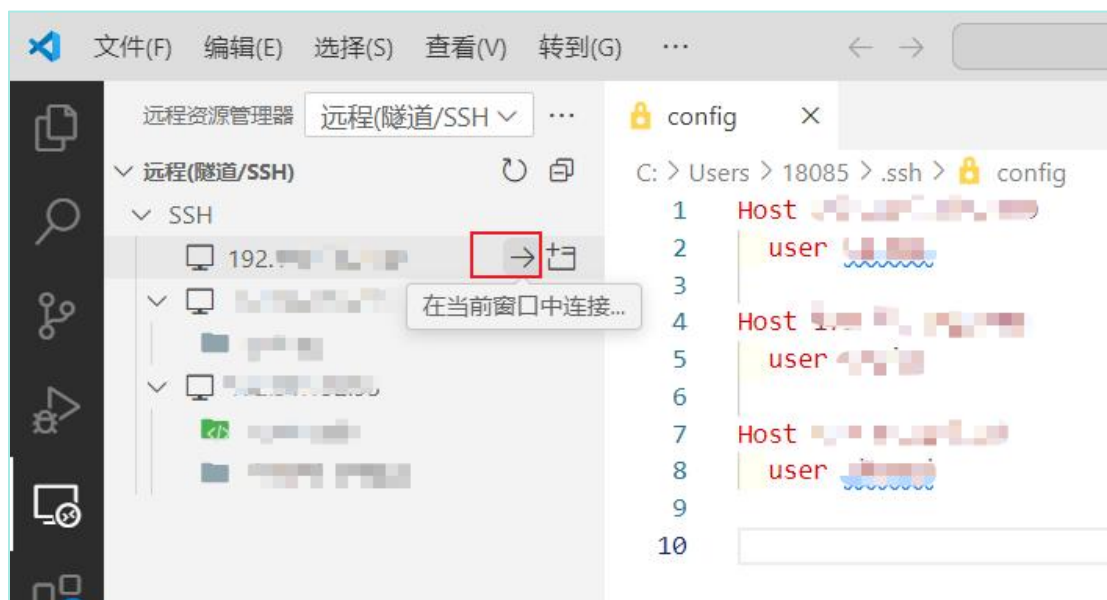
然后依次点击



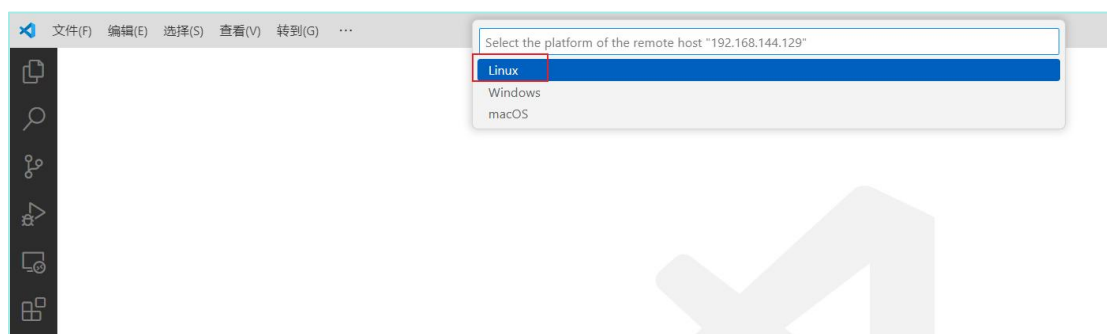
就可以打开配置文件，然后写上 IP 地址和用户名，其他的可以删掉。你有多个服务器就可以往下写就行：



填写完左边就会出来连接的箭头了：



然后选择你的服务器类型，一般都是 linux 哈：



然后输入密码就好啦！

本地电脑终端连接服务器方法，ssh 用户名@IP 地址，然后输入密码（不会显示哈）就好了：

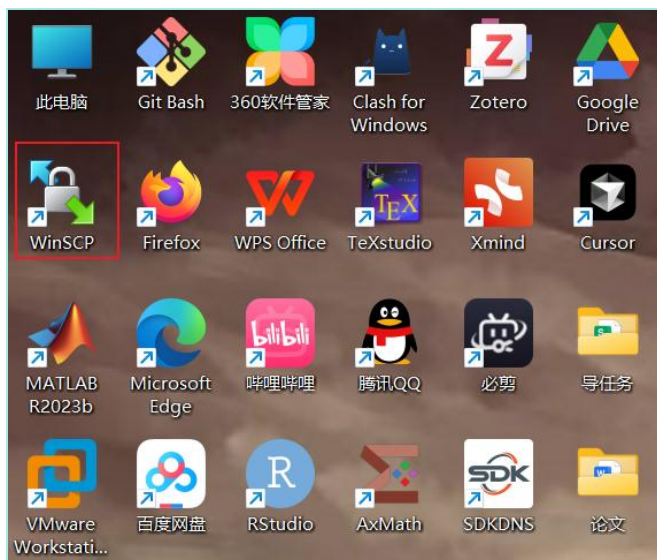
```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

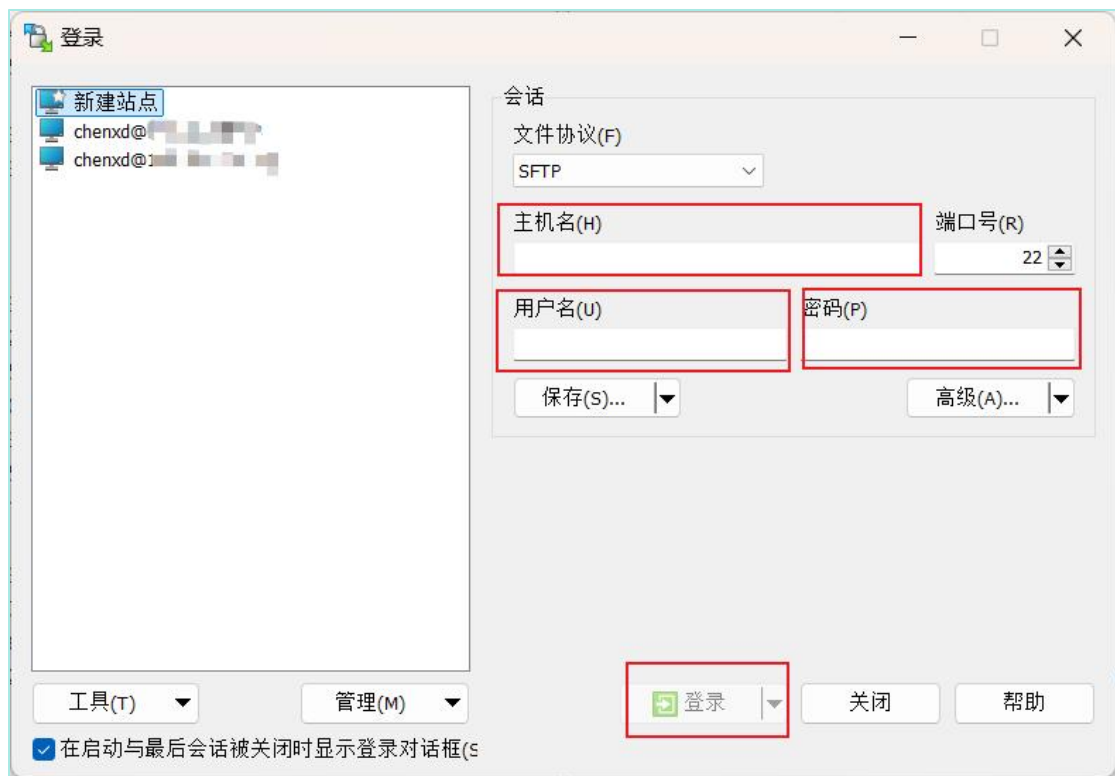
加载个人及系统配置文件用了 569 毫秒。
(base) PS C:\Users\18085> ssh chenxd@192.168.144.129
chenxd@192.168.144.129's password:

Last login: Wed Feb 26 01:12:27 2025 from 192.168.144.129
(base) chenxd@userh3c-R5350-G6:~$ ls -la
. .bash_history .bashrc .conda .dotnet lwfx .profile snap .vscode-server
.. .bash_logout .cache .config .local .nv .python_history .viminfo .wget-hsts
(base) chenxd@userh3c-R5350-G6:~$
```

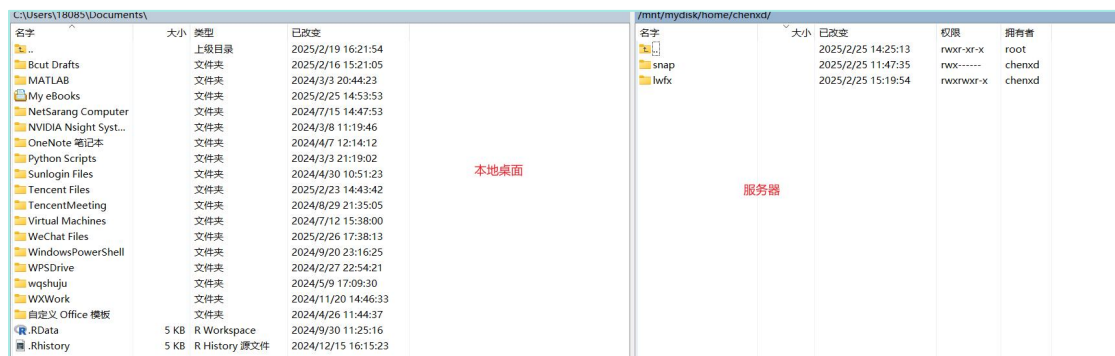
如果你用不惯终端，那么下面这个软件可以帮你（网页直接下载即可）



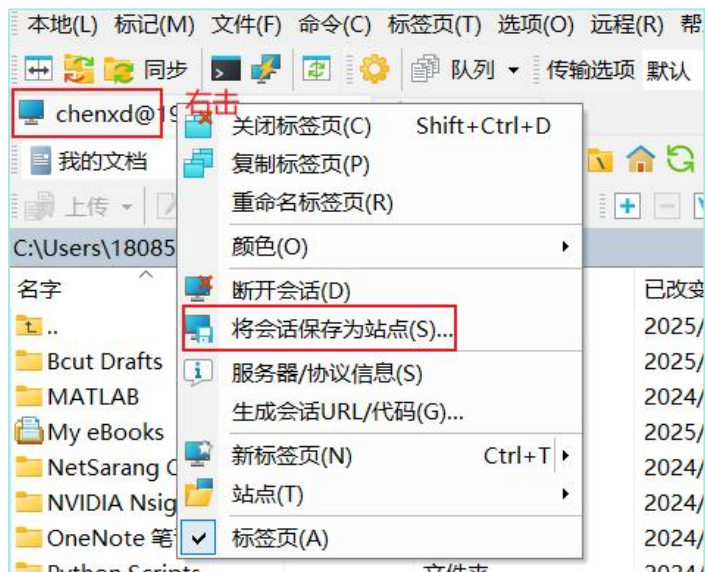
填上相关信息就好啦：



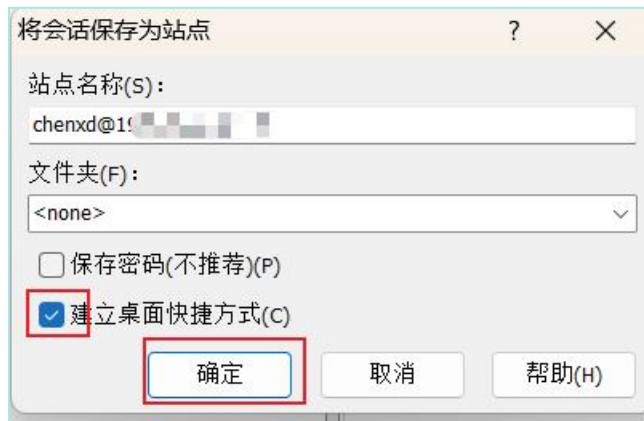
你就可以直接通过“拖拉拽”将本地和服务进行文件互通啦：



当然我们也可以保存为桌面快捷方式：



点击



以后直接点这个就可以啦！（如果电脑是私人的，可以把“保存密码”也勾选上）。

2.8 linux 配置 rpy2 模块

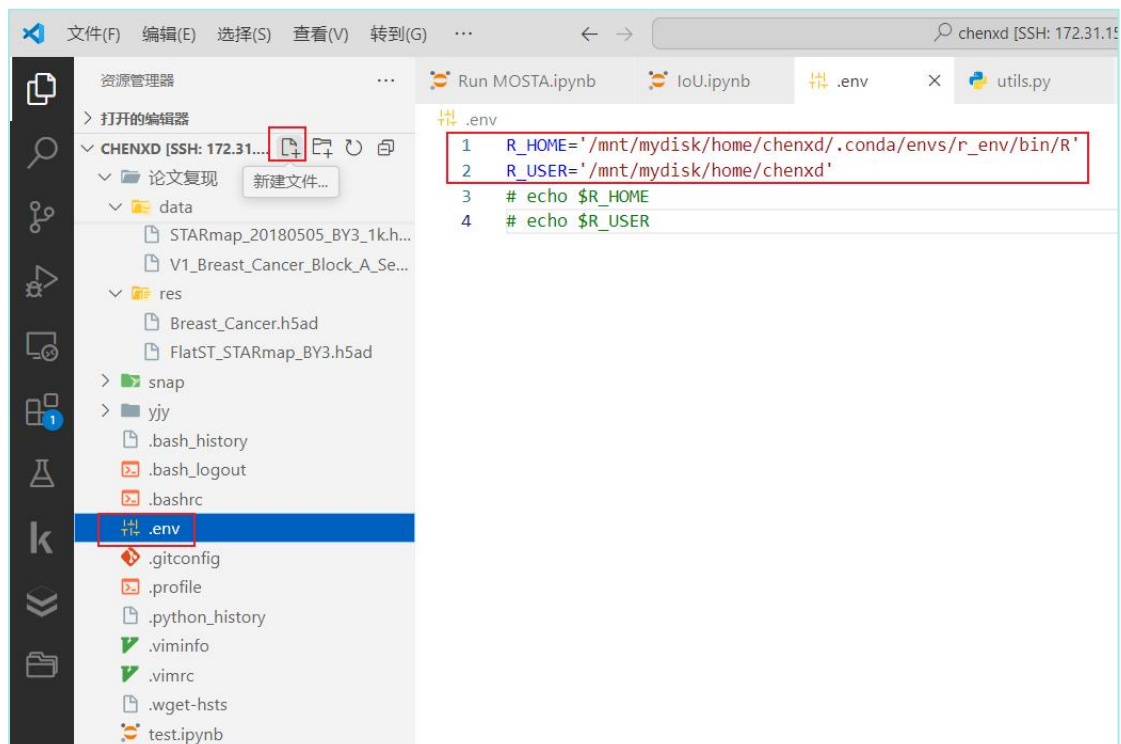
有时候我们需要在 Python 的虚拟环境中调用 R 的模块。不会还有人和我一样去真的单独装 R 吧哈哈哈，其实利用 conda 创建一个虚拟环境就好啦！连接好服务器后，在终端输入（r_env 是虚拟环境的名字，可以改）：

```
conda create -n r_env r=4.3
```

然后我们进入 r 的虚拟环境，查看 r 的位置：

```
(base) chenxd@userh3c-R5350-G6:~$ conda activate r_env
(r_env) chenxd@userh3c-R5350-G6:~$ which R
/mnt/mydisk/home/chenxd/.conda/envs/r_env/bin/R
(r_env) chenxd@userh3c-R5350-G6:~$
```

接下来配置 r 语言环境变量，新建一个 .env 的文件，配置如下命令：



好啦，我们在 Python 环境中测试一下是否可以安装并导入 rpy2:

```
(STAGATE) chenxd@userh3c-R5350-G6:~$ pip install rpy2
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: rpy2 in ./conda/envs/STAGATE
Requirement already satisfied: cffi>=1.15.1 in ./conda/envs/STAGATE
Requirement already satisfied: Jinja2 in ./conda/envs/STAGATE
Requirement already satisfied: tzlocal in ./conda/envs/STAGATE
Requirement already satisfied: pycparser in ./conda/envs/STAGATE
(2.22)
Requirement already satisfied: MarkupSafe>=2.0 in ./conda/envs/STAGATE
(2.1.5)
(STAGATE) chenxd@userh3c-R5350-G6:~$ python
Python 3.9.0 (default, Nov 15 2020, 14:28:56)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license()" for more
>>> import rpy2
>>>
```

如果不报错就说明没问题了。

如果遇到安装 rpy2 报错说环境的问题 bulabula, 那么请首先输入:

```
export PATH=$PATH:/mnt/mydisk/home/chenxd/.conda/envs/r_env/bin
```

将自己的 R 位置添加到环境变量里面。

下一步，我们需要在 R 语言环境中去安装我们要用到的包，这里以 `mclust` 为例（下面步骤我就不截图了）：

step1: 切换到 r 的虚拟环境: `conda activate r env;`

step2: 输入 'R' 进入 r 编译器;

step3: 安装需要的包: `install.packages("mclust")`, 然后选择来源就好了 (比如中国深圳就是 23);

最后来验证一下 `mclust` 是否可用:

```
1 adata = STAGATE_pyG.mclust_R(adata, used_obs='STAGATE', num_cluster=7)
```

[24] ✓ 5.1s

```
... R[write to console]:
```

```

  _____/ / _____/ /
 / _ _ \ _ _ / / / / / / _ _ \
 / / / / / / / / / / / / ( _ ) /
 / / / / / _ \ / _ \ / _ \ / _ \ version 6.1.1
Type 'citation("mclust")' for citing this R package in publications.

fitting ...
|
|=====| 0%
|=====| 100%
```

可以发现已经没问题了！

如果还是报错，内核崩溃，存在和我下列一样的问题，请测试一下：

```
import rpy2.robjects as robjects
```

如果反馈是：


```
Unable to determine R library path: [Errno 2] No such file or directory: '/home/chenxd/.conda/envs/r_env/lib/R/bin/Rscript'
Fatal error: unable to open the base package
```

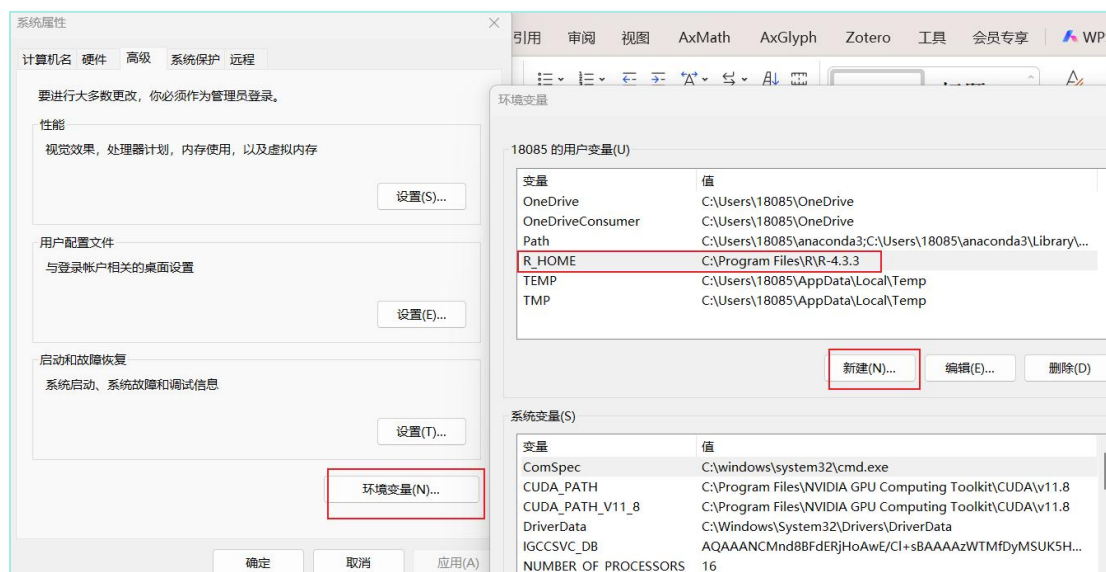
... 在当前单元格或上一个单元格中执行代码时 **Kernel 崩溃**。
 请查看单元格中的代码，以确定故障的可能原因。
 单击[此处](#)了解详细信息。
 有关更多详细信息，请查看 [Jupyter log](#)。

那么请你在你的代码前面手动加上（对应自己的 R 环境路径）：

```
import os
```

```
os.environ['R_HOME'] = '/mnt/mydisk/home/chenxd/.conda/envs/r_env/lib/R'
```

Windows 端一定要新建一个 R_HOME 对应到你的 R 哈：



然后再代码运行前设置这两个参数就好啦！（找到自己电脑对应的位置）

```
os.environ['R_USER']=
```

```
r'C:\Users\18085\anaconda3\envs\torch3\Lib\site-packages\rpy2'
```

```
os.environ['R_HOME'] = 'C:\Program Files\R\R-4.3.3'
```

2.9 github 项目包配置（setup.py 方法）

github 项目通常有好几种方式配置，我用过的有直接 **pip** 安装（这种比较少）、通过 **yml** 或者 **yaml** 或者 **requiremen.txt** 进行安装，或者放在同一项目直接 **import**，或者使用 **Docker** 容器，最好用的还是今天分享的 **setup.py** 方法。

首先打开 **github** 项目，查看其是否有这个文件，如果有就可以用这种方式进行配置：

 QIFEIDKN	Update README.md	d6dd9f4 · 3 years ago	🕒 17 Commits
📁 STAGATE	v1.0.1 update	3 years ago	
📄 LICENSE	Initial commit	4 years ago	
📄 README.md	Update README.md	3 years ago	
📄 README.rst	Add files via upload	4 years ago	
📄 STAGATE_Overview.png	Add files via upload	4 years ago	
📄 requirement.txt	Add files via upload	4 years ago	
📄 setup.py	v1.0.1 update	3 years ago	

然后我们打开 readme 文件,看这个项目使用的框架,可以看到这个项目是 pytorch 框架:

Files
main
Go to file
STAGATE_pyG
LICENSE
README.md
README.rst
STAGATE_Overview.png
requirement.txt
setup.py

STAGATE_pyG / README.md
Preview Code Blame 44 lines (27 loc) · 2.31 KB

the latent embeddings and the reconstructed expression profiles can be visualization, spatial trajectory inference, data denoising and 3D express

STAGATE based on tensorflow framework can be found at [here](#).

Getting started

See [Documentation and Tutorials](#).

Software dependencies

scanpy

pytorch

pyG

step1: 现在进入你想要使用的虚拟环境和路径:

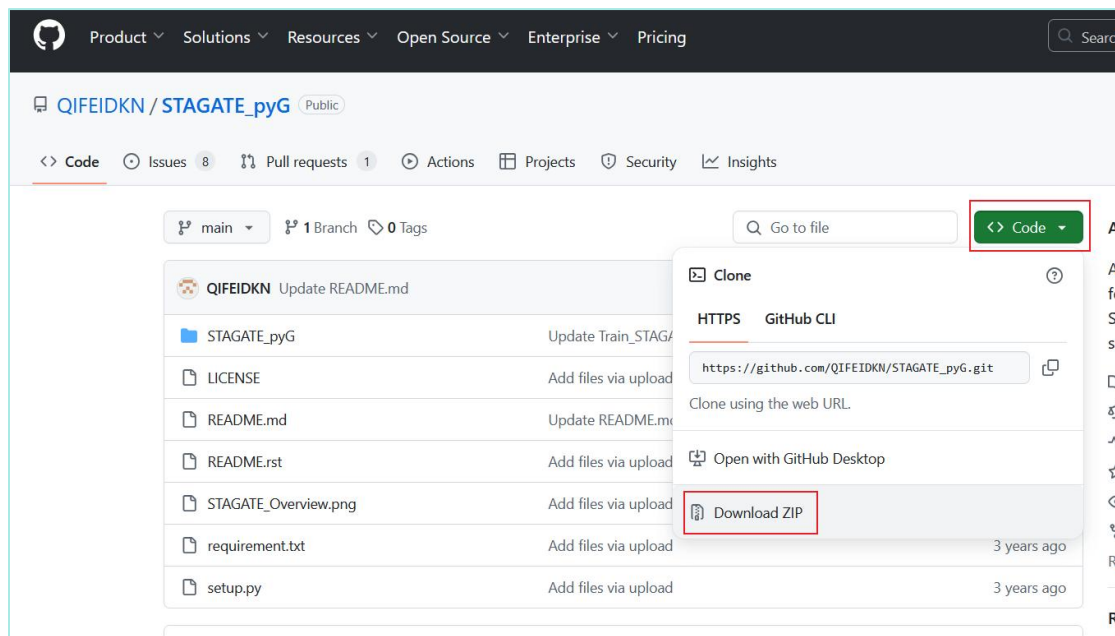
```
(base) chenxd@userh3c-R5350-G6:~$ conda activate STAGATE
(STAGATE) chenxd@userh3c-R5350-G6:~$ cd lwfx/1_STAGATE/code
(STAGATE) chenxd@userh3c-R5350-G6:~/lwfx/1_STAGATE/code$
```

step2: 然后我们就要获取 github 项目包,我们可以直接使用 git clone 命令:

Product Solutions Resources Open Source Enterprise Pricing
QIFEIDKN / STAGATE_pyG Public
Code Issues 8 Pull requests 1 Actions Projects Security Insights
main 1 Branch 0 Tags
Go to file
Code
Clone
HTTPS GitHub CLI
Copy url to clipboard
https://github.com/QIFEIDKN/STAGATE_pyG.git
Clone using the web URL
Open with GitHub Desktop
Download ZIP

```
(STAGATE) chenxd@userh3c-R5350-G6:~/Lwfx/1_STAGATE/code$ git clone https://github.com/QIFEIDKN/STAGATE_pyG.git
Cloning into 'STAGATE_pyG'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 62 (delta 32), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (62/62), 1.60 MiB | 2.99 MiB/s, done.
Resolving deltas: 100% (32/32), done.
(STAGATE) chenxd@userh3c-R5350-G6:~/Lwfx/1_STAGATE/code$
```

step2 (备选)：如果上面不成功，我们也可以直接下载压缩包后解压，然后在通过 vscode 或者 WinSCP 转移到服务器中：



step3: 打开 setup.py，一定要确定有 `zip_safe = False`，我们难免有修改别人代码的情况，这个指令能让我们轻松修改源码。

```
/mnt/mydisk/home/chenxd/lwfx/1_STAGATE/code/STAGATE_pyG/setup.py - chenxd@192.168.144.129 - 编辑器 - WinSCP
from setuptools import Command, find_packages, setup

__lib_name__ = "STAGATE_pyG"
__lib_version__ = "1.0.0"
__description__ = "Deciphering spatial domains from spatially resolved transcriptomics with adaptive g
__url__ = "https://github.com/QIFEIDKN/STAGATE"
__author__ = "Kangning Dong"
__author_email__ = "dongkangning16@mails.ucas.ac.cn"
__license__ = "MIT"
__keywords__ = ["spatial transcriptomics", "Deep learning", "Graph attention auto-encoder"]
__requires__ = ["requests",]

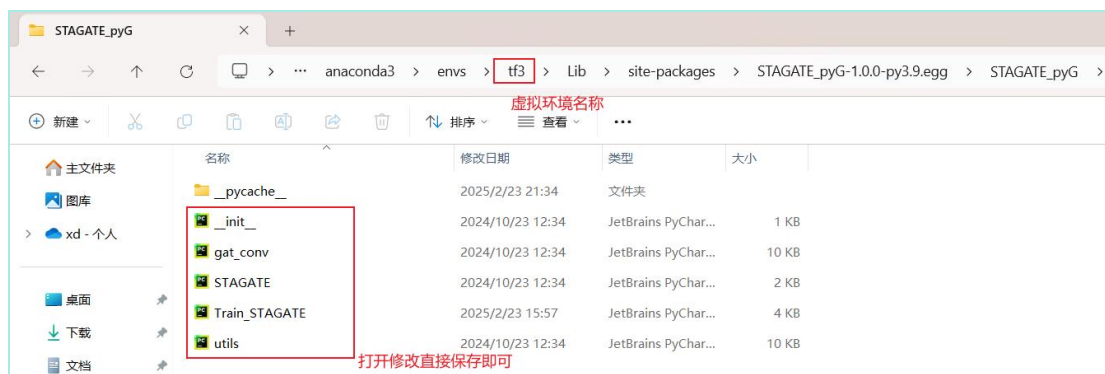
with open("README.rst", "r", encoding="utf-8") as f:
    __long_description__ = f.read()

setup(
    name = __lib_name__,
    version = __lib_version__,
    description = __description__,
    url = __url__,
    author = __author__,
    author_email = __author_email__,
    license = __license__,
    packages = ['STAGATE_pyG'],
    install_requires = __requires__,
    zip_safe = False,
    include_package_data = True,
    long_description = __long_description__
)
```

step3（续）：如何找到配置的软件包并修改

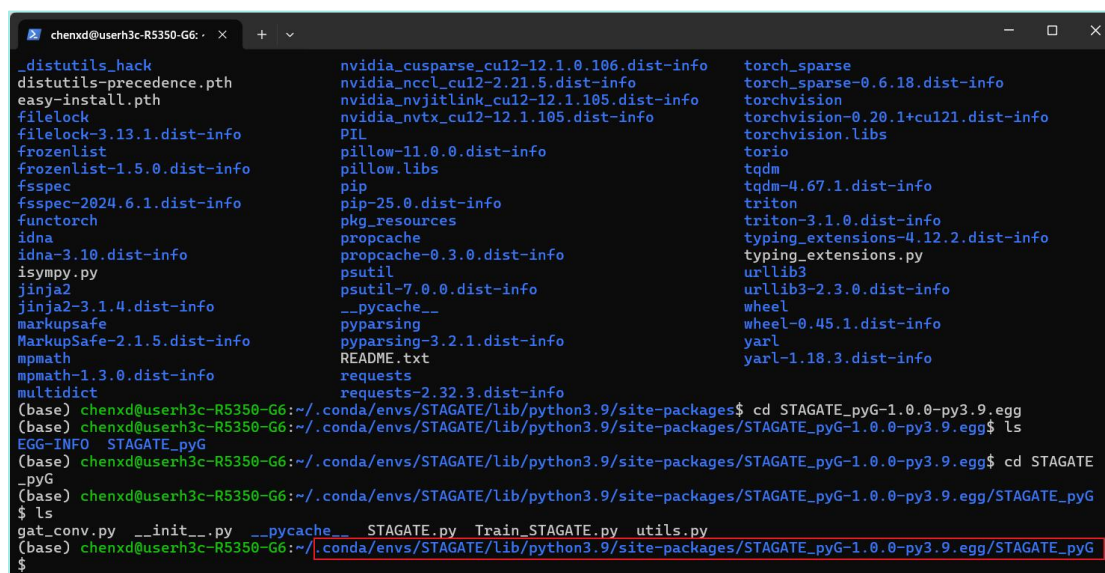
Windows 一般源码路径：

C:\Users\18085\anaconda3\envs\tf3\Lib\site-packages\STAGATE_pyG-1.0.0-py3.9.egg\STAGATE_pyG

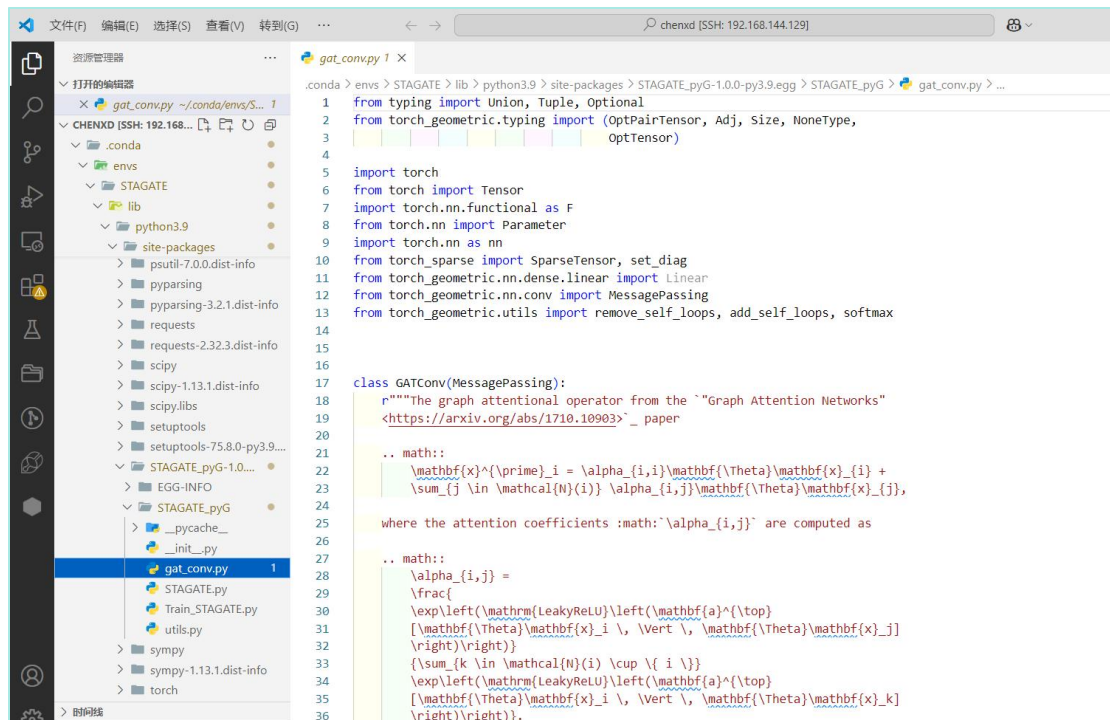


linux 一般源码路径：

.conda/envs/<环境名>/lib/python<版本号>/site-packages/



这样我们就可以通过 vs code 编辑源文件了。



```

1 from typing import Union, Tuple, Optional
2 from torch_geometric.typing import (OptPairTensor, Adj, Size, NoneType,
3                                     OptTensor)
4
5 import torch
6 from torch import Tensor
7 import torch.nn.functional as F
8 from torch.nn import Parameter
9 import torch.nn as nn
10 from torch_sparse import SparseTensor, set_diag
11 from torch_geometric.nn.dense.linear import Linear
12 from torch_geometric.nn.conv import MessagePassing
13 from torch_geometric.utils import remove_self_loops, add_self_loops, softmax
14
15
16
17 class GATConv(MessagePassing):
18     r"""The graph attentional operator from the "Graph Attention Networks"
19     <https://arxiv.org/abs/1710.10903> paper
20
21     .. math::
22         \mathbf{x}^{\prime}_i = \alpha_{i,i} \mathbf{\Theta} \mathbf{x}_i +
23         \sum_{j \in \mathcal{N}(i)} \alpha_{i,j} \mathbf{\Theta} \mathbf{x}_j,
24
25     where the attention coefficients :math:`\alpha_{i,j}` are computed as
26
27     .. math::
28         \alpha_{i,j} =
29         \frac{
30             \exp\left(\mathbf{a}^{\top} \left[ \mathbf{\Theta} \mathbf{x}_i \parallel \mathbf{\Theta} \mathbf{x}_j \right] \right)
31         }{
32             \sum_{k \in \mathcal{N}(i)} \exp\left(\mathbf{a}^{\top} \left[ \mathbf{\Theta} \mathbf{x}_i \parallel \mathbf{\Theta} \mathbf{x}_k \right] \right)
33         }
34
35     """
36

```

step4: cd 到 setup.py 的路径:

```

(STAGATE) chenxd@userh3c-R5350-G6:~/lwf/1_STAGATE/code$ git clone https://github.com/QIFEIDKN/
Cloning into 'STAGATE_pyG'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 62 (delta 32), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (62/62), 1.60 MiB | 2.99 MiB/s, done.
Resolving deltas: 100% (32/32), done.
(STAGATE) chenxd@userh3c-R5350-G6:~/lwf/1_STAGATE/code$ ls
STAGATE_pyG
(STAGATE) chenxd@userh3c-R5350-G6:~/lwf/1_STAGATE/code$ cd STAGATE_pyG
(STAGATE) chenxd@userh3c-R5350-G6:~/lwf/1_STAGATE/code/STAGATE_pyG$ ls
LICENSE README.md README.rst requirement.txt setup.py STAGATE_Overview.png STAGATE_pyG
(STAGATE) chenxd@userh3c-R5350-G6:~/lwf/1_STAGATE/code/STAGATE_pyG$

```

step5: 输入配置指令

首先安装 requests 以防下方报错:

pip install requests

```

chenxd@userh3c-R5350-G6: ~
File "<frozen importlib._bootstrap>", line 986, in _find_and_load_unlocked
File "<frozen importlib._bootstrap>", line 680, in _load_unlocked
File "<frozen importlib._bootstrap_external>", line 790, in exec_module
File "<frozen importlib._bootstrap>", line 228, in _call_with_frames_removed
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/setuptools/command/easy_install.py", line 41, in <module>
import pkg_resources
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 3657, in <module>
def _initialize_master_working_set() -> None:
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 3640, in _call_aside
f(*args, **kwargs)
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 3669, in _initialize_master_working_set
working_set = _declare_state('object', 'working_set', WorkingSet._build_master())
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 654, in _build_master
ws.require(_requires_)
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 1061, in require
needed = self.resolve(parse_requirements(requirements))
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 888, in resolve
dist = self._resolve_dist(
File "/mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages/pkg_resources/__init__.py", line 929, in _resolve_dist
raise DistributionNotFound(req, requirers)
pkg_resources.DistributionNotFound: The 'requests' distribution was not found and is required by the application
(STAGATE) chenxd@userh3c-R5350-G6: ~/Lwfx/1_STAGATE/code/STAGATE_pyG$

```

然后依次输入：

python setup.py build

python setup.py install

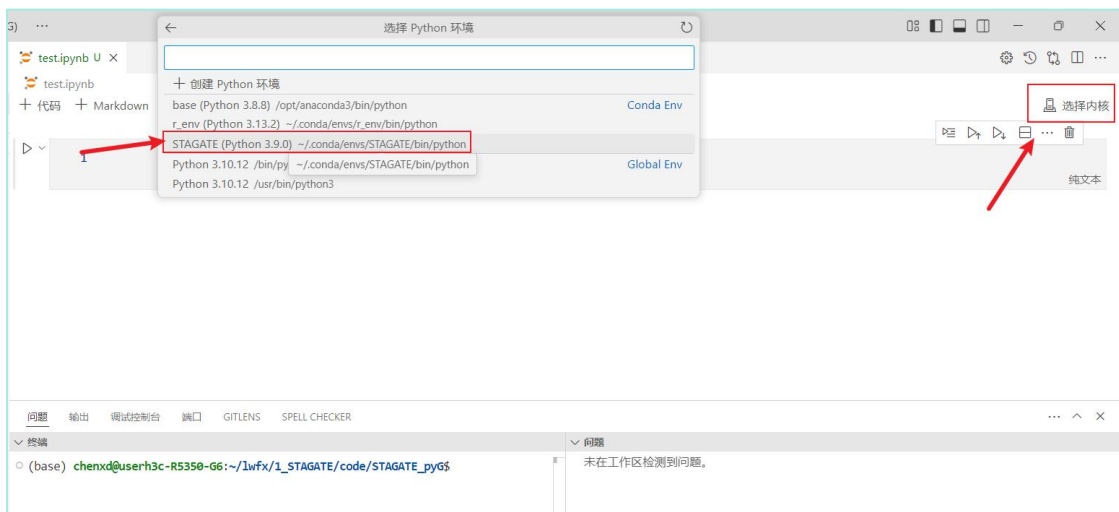
最后我们可以测试一下是否配置成功

```

Using /mnt/mydisk/home/chenxd/.conda/envs/STAGATE/lib/python3.9/site-packages
Finished processing dependencies for STAGATE-pyG==1.0.0 成功
(STAGATE) chenxd@userh3c-R5350-G6: ~/Lwfx/1_STAGATE/code/STAGATE_pyG$ python
Python 3.9.0 (default, Nov 15 2020, 14:28:56)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import STAGATE_pyG
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/mnt/mydisk/home/chenxd/Lwfx/1_STAGATE/code/STAGATE_pyG/STAGATE_pyG/__init__.py", line 11, in <module>
    from .STAGATE import STAGATE
  File "/mnt/mydisk/home/chenxd/Lwfx/1_STAGATE/code/STAGATE_pyG/STAGATE_pyG/STAGATE.py", line 9, in <module>
    from .gat_conv import GATConv
  File "/mnt/mydisk/home/chenxd/Lwfx/1_STAGATE/code/STAGATE_pyG/STAGATE_pyG/gat_conv.py", line 2, in <module>
    from torch_geometric.typing import OptPairTensor, Adj, Size, NoneType,
ModuleNotFoundError: No module named 'torch_geometric'
>>>

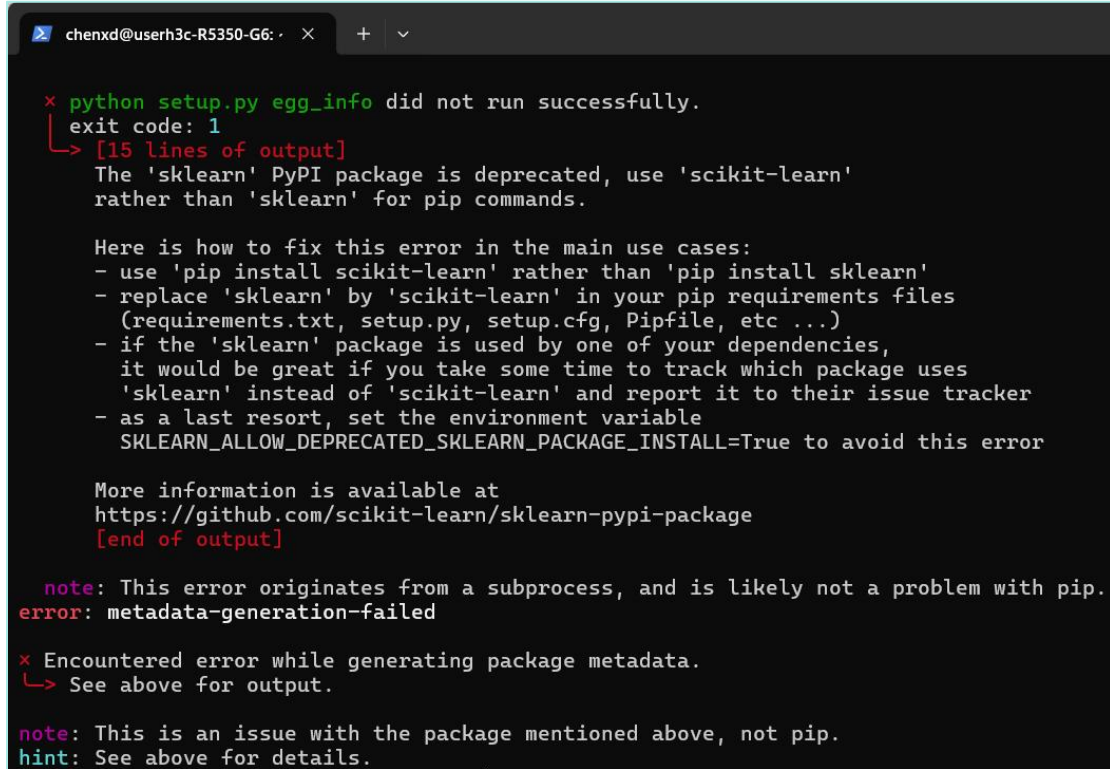
```

可以看到已经成功配置，上面那个报错只是因为当前虚拟环境缺少这个模块，直接 pip 安装就好啦！好了，接下来你就可以在 vs code 中使用这个虚拟环境去运行项目代码了。



2.10 虚拟环境调试

当我们在安装模块的时候出现下列这种报错“xxxx, not pip”（这个报错的原因是因为包的名字写错了！），一般都是虚拟环境中各个模块之间版本不匹配导致装包不成功：



```

chenxd@userh3c-R5350-G6: ~$ python setup.py egg_info did not run successfully.
exit code: 1
[15 lines of output]
The 'sklearn' PyPI package is deprecated, use 'scikit-learn'
rather than 'sklearn' for pip commands.

Here is how to fix this error in the main use cases:
- use 'pip install scikit-learn' rather than 'pip install sklearn'
- replace 'sklearn' by 'scikit-learn' in your pip requirements files
  (requirements.txt, setup.py, setup.cfg, Pipfile, etc ...)
- if the 'sklearn' package is used by one of your dependencies,
  it would be great if you take some time to track which package uses
  'sklearn' instead of 'scikit-learn' and report it to their issue tracker
- as a last resort, set the environment variable
  SKLEARN_ALLOW_DEPRECATED_SKLEARN_PACKAGE_INSTALL=True to avoid this error

More information is available at
https://github.com/scikit-learn/sklearn-pypi-package
[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
error: metadata-generation-failed

× Encountered error while generating package metadata.
→ See above for output.

note: This is an issue with the package mentioned above, not pip.
hint: See above for details.

```

容易版本不匹配的包：Scanpy、stlearn（这个包要注意，我们通过 pip 安装的是 scikit-learn，import 的是 sklearn）、numpy、numba 等；那么应该怎么去安装版本匹配的版本呢？直接在一起 pip 即可，pip 会自动帮助我们筛选各个相适应的版本：

pip install Scanpy scikit-learn numpy numba

如果还是不行，可以先卸载 pip uninstall 那些“钉子包”，然后再装。

如果需要更改某一个模块的版本，我们可以利用下列命令快速查找模块版本

pip install numpy ==

上列命令有些服务器终端不支持，可以随便写一个版本，如下：

pip install numpy==10.2

```
(STAGATE) chenxd@userh3c-R5350-G6:~$ pip install numpy==10.2
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
ERROR: Could not find a version that satisfies the requirement
10.2.0, 1.6.1, 1.6.2, 1.7.0, 1.7.1, 1.7.2, 1.8.0, 1.8.1, 1.8.2, 1
1.10.4, 1.11.0, 1.11.1, 1.11.2, 1.11.3, 1.12.0, 1.12.1, 1.13.0,
1.14.5, 1.14.6, 1.15.0, 1.15.1, 1.15.2, 1.15.3, 1.15.4, 1.16.0,
1.17.1, 1.17.2, 1.17.3, 1.17.4, 1.17.5, 1.18.0, 1.18.1, 1.18.2,
1.19.4, 1.19.5, 1.20.0, 1.20.1, 1.20.2, 1.20.3, 1.21.0, 1.21.1,
1.22.2, 1.22.3, 1.22.4, 1.23.0, 1.23.1, 1.23.2, 1.23.3, 1.23.4,
1.25.1, 1.25.2, 1.26.0, 1.26.1, 1.26.2, 1.26.3, 1.26.4, 2.0.0,
ERROR: No matching distribution found for numpy==10.2
(STAGATE) chenxd@userh3c-R5350-G6:~$ |
```

这样我们就可以根据对应包的版本一个个尝试了。

2.11 已经安装了某个包 import 还是报错解决方案

问题描述，在 vs code import 报错：

```
----> 8 import GraphST
      9 import tracemalloc
     10 import time

File ~/lwfx/4_GraphST/code/GraphST/__init__.py:11
      8 __author__ = "Yahui Long"
      9 __email__ = "long_yahui@immunol.a-star.edu.sg"
----> 11 from .utils import clustering
     12 from .preprocess import preprocess_adj, preprocess, constru

ModuleNotFoundError: No module named 'GraphST.utils'
```

但是在终端却没有问题：

```
○ (graphst) (base) chenxd@userh3c-R5350-G6:~/lwfx/4_GraphST/code/GraphST$ python
Python 3.9.0 (default, Nov 15 2020, 14:28:56)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import GraphST
>>> |
```

以下提供几种思路，大家自己对应去检查：

1. 首先确保适用相同的虚拟环境，这个没啥好说的；
2. 确保 python 解释器是一致的：

```

1 import sys
2 print(sys.executable)

```

[8] ✓ 0.0s

... /mnt/mydisk/home/chenxd/.conda/envs/graphst/bin/python

问题 109 输出 调试控制台 端口 SPELL CHECKER 107 JUPYTER

终端

• (graphst) (base) chenxd@userh3c-R5350-G6:~/lwf/4_GraphST/code/GraphST\$ which python
 /mnt/mydisk/home/chenxd/.conda/envs/graphst/bin/python

○ (graphst) (base) chenxd@userh3c-R5350-G6:~/lwf/4_GraphST/code/GraphST\$

vs code 是全自动的，所以不用太担心。如果是本地 jupyter 发生这种情况，需要切换虚拟环境内核，步骤有点繁琐，详情见 b 站视频“已经装了某个包，jupyter notebook 还是说 No module named 'xx'解决方法”；

https://www.bilibili.com/video/BV13oB4YPEyk/?vd_source=ff01904c2f07c3b212c1669111fca268

3.查看 jupyter 内核是否和终端一致：

```

1 !jupyter kernelspec list

```

[3] ✓ 0.6s

... Available kernels:
 python3 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/share/jupyter/kernels/python3

• (GraphST1) chenxd@userh3c-R5350-G6:~/lwf/4_GraphST/code/GraphST\$ jupyter kernelspec list
 Available kernels:
 python3 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/share/jupyter/kernels/python3

○ (GraphST1) chenxd@userh3c-R5350-G6:~/lwf/4_GraphST/code/GraphST\$

4.将模块所在位置添加到工作目录中 sys.path.append:

```

1 import sys
2 for i in range(0, len(sys.path)):
3     print(sys.path[i])

```

✓ 0.0s

/mnt/mydisk/home/chenxd/.conda/envs/GraphST1/lib/python3.11.zip
 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/lib/python3.11
 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/lib/python3.11/lib-dynload
 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/lib/python3.11/site-packages
 /mnt/mydisk/home/chenxd/.conda/envs/GraphST1/lib/python3.11/site-packages/GraphST-1.1.1-py3.11.egg

5.其他冲突（大家一般用不上）

根目录名包含了一个 __init__.py 文件，这个真的把我坑死了。虽然 Python 默认不会将带有非法字符（如连字符 -）的目录识别为包，但某些编辑器（如 VS Code、PyCharm）或开发工具会尝试将项目根目录隐式添加到 sys.path，导致 Python 试图将其作为包导入，从而引发混乱。

终端正常但编辑器报错：本质是编辑器隐式添加了项目根目录到 sys.path，而根目录的 __init__.py 和非法名称导致路径冲突。

2.12 去修改 github 源码解决报错

今天遇到了这样一个问题，明明几个月前都能运行的 github 包，但是今天来运行都报错：

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[6], line 2
      1 # Calculate PI
----> 2 adata = PROST.prepare_for_PI(adata, platform=██████████)
      3 adata = PROST.cal_PI(adata, platform="visium")
      5 # Calculate spatial autocorrelation statistics and do hypothesis test

File ~/conda/envs/prost/lib/python3.11/site-packages/PROST-1.1.2-py3.11.egg/PROST/calculate_PI.py:
   14 def prepare_for_PI(adata, grid_size=20, percentage=0.1, platform="visium"):
--> 15     selected_gene_idx, postcount = pre_process(adata, percentage, var_stabilization = False)
   17     if platform=="visium" or platform=="ST":
   18         try:

File ~/conda/envs/prost/lib/python3.11/site-packages/PROST-1.1.2-py3.11.egg/PROST/utis.py:266, in
   243 """
   244 Pre-process gene count.
   245
   (...) 263 Expression matrix of genes that `percentage` greater than threshold.
   264 """
   265 if sp.issparse(adata.X):
--> 266     rawcount = adata.X.A.T
   267 else:
   268     rawcount = adata.X.T

AttributeError: 'csr matrix' object has no attribute 'A'
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[6], line 50
   48 r_seed=t_seed=n_seed=100
   49 #Seaech for suitable resolution
--> 50 res=spg.search_res(adata, adj, l, n_clusters, start=0.7, step=0.1, tol=5e-3, lr=0.05, max_epochs=
   52 #run SpaGCN
   53 clf=spg.SpaGCN()

File ~/conda/envs/spagcn/lib/python3.11/site-packages/SpaGCN-1.2.7-py3.11.egg/SpaGCN/util.py:353, in sea
   351 clf=SpaGCN()
   352 clf.set_l(l)
--> 353 clf.train(adata,adj,init_spa=True,init=██████████,res=res, tol=tol, lr=lr, max_epochs=max_epochs)
   354 y_pred, _=clf.predict()
   355 old_num=len(set(y_pred))

File ~/conda/envs/spagcn/lib/python3.11/site-packages/SpaGCN-1.2.7-py3.11.egg/SpaGCN/SpaGCN.py:51, in Sp
   49 pca = PCA(n_components=self.num_pcs)
   50 if issparse(adata.X):
--> 51     pca.fit(adata.X.A)
   52     embed=pca.transform(adata.X.A)
   53 else:

AttributeError: 'csr_matrix' object has no attribute 'A'
```


把错误复制到豆包，给的反馈是因为版本更新了，一些方法不再支持了：

报错原因

在较新的 `scipy` 版本里，`csr_matrix` 对象不再支持 `A` 属性。`A` 属性原本用于将稀疏矩阵转换为密集矩阵，不过现在建议使用 `toarray()` 方法来达成此目的。

解决方案

你需要修改 `PROST` 库中 `utils.py` 文件里的代码，把 `rawcount = adata.X.A.T` 替换成 `rawcount = adata.X.toarray().T`。

以下是修改后的代码示例：

```
python ^
# 在 utils.py 文件中
if sp.issparse(adata.X):
    rawcount = adata.X.toarray().T
else:
    rawcount = adata.X.T
```

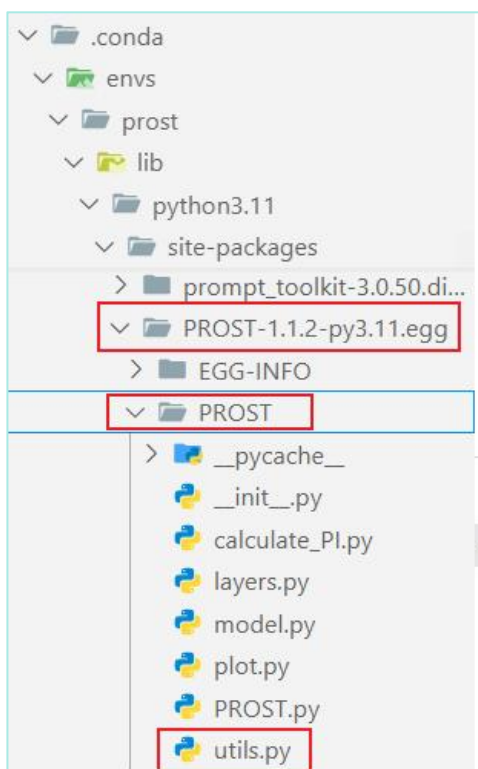
以下提供两种解决方法，第一种就是把报错的那个软件包安装旧版本的，这种是非常不建议的，我们可以 `pip show` 一下，可以看到其牵连的包非常多，往往修改一个就要修改多个。

Requires: numpy
Required-by: anndata, pynndescent, scanpy, scikit-image, scikit-learn, statsmodels, umap-learn

那么接下来，我们就采纳豆包的建议，去源码里面修改。首先找到源码所在位置，一般 `linux` 的位置在：

用户名/.conda/envs/虚拟环境名/lib/python3.XX/site-packages

找到我们在 `github` 安装的包：



打开功效函数，进行修改把 `adata.X.A.T` 都替换成 `rawcount = adata.X.toarray().T`

```

242 def pre_process(adata, percentage = 0.1, var_stabilization = True):
260     gene_use
261     Index of genes that `percentage` greater than threshold.
262     rawcount
263     Expression matrix of genes that `percentage` greater than threshold.
264     """
265     if sp.issparse(adata.X):
266         rawcount = adata.X.toarray().T#adata.X.A.T
267     else:
268         rawcount = adata.X.T

```

我这边建议的是将源码进行注释修改，而不是直接删除，因为有时候我们一次修改不成功，可能还需要回来重新修改。每次更新源码后，我们都需要重启内核才能生效：

The screenshot shows a Jupyter Notebook window titled "1Run_DLPCF.ipynb". The code cell contains imports for pandas, numpy, scanpy, os, warnings, matplotlib, and PROST. The output shows the PROST version as 1.1.2. A "Restart" button is highlighted in the toolbar. A dialog box from Visual Studio Code asks "是否要重启 Jupyter kernel? 所有变量都将丢失。" (Do you want to restart the Jupyter kernel? All variables will be lost.) with buttons for "重启" (Restart), "不再询问" (Don't ask again), and "取消" (Cancel). The "Restart" button is highlighted.

现在再次重新运行之前报错的代码：

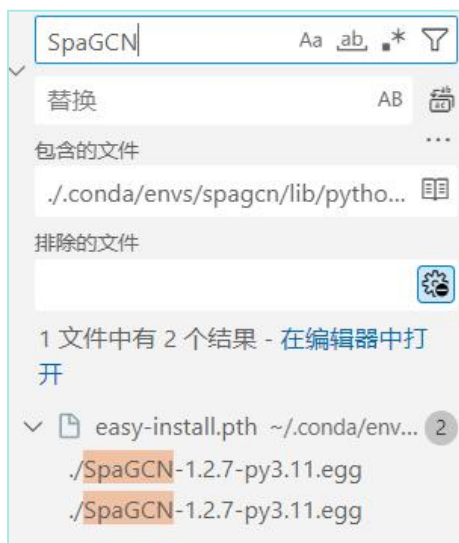
```

... Calculateing adj matrix using xy only...
Run 1: l [0.01, 1000], p [0.0, 304.12201744612213]
Run 2: l [0.01, 500.005], p [0.0, 83.85807659348846]
Run 3: l [0.01, 250.0075], p [0.0, 21.34114436831361]
Run 4: l [0.01, 125.00874999999999], p [0.0, 4.750619747568681]
Run 5: l [0.01, 62.509375], p [0.0, 0.5111031977572873]
Run 6: l [31.2596875, 62.509375], p [0.00033941474562015905, 0.5111031977572873]
Run 7: l [46.884531249999995, 62.509375], p [0.07618062809320492, 0.5111031977572873]
Run 8: l [54.69695312499999, 62.509375], p [0.2408340503002373, 0.5111031977572873]
Run 9: l [58.6031640625, 62.509375], p [0.3638374517377281, 0.5111031977572873]
Run 10: l [60.556269531249995, 62.509375], p [0.4346341140578587, 0.5111031977572873]
Run 11: l [61.532822265625, 62.509375], p [0.4721853648227945, 0.5111031977572873]
recommended l = 62.021098632812496

```

然后就好啦！

有的同学可能遇到下列这种情况，找不到包怎么办？然后搜索发现可以搜索到，但是不是文件夹：



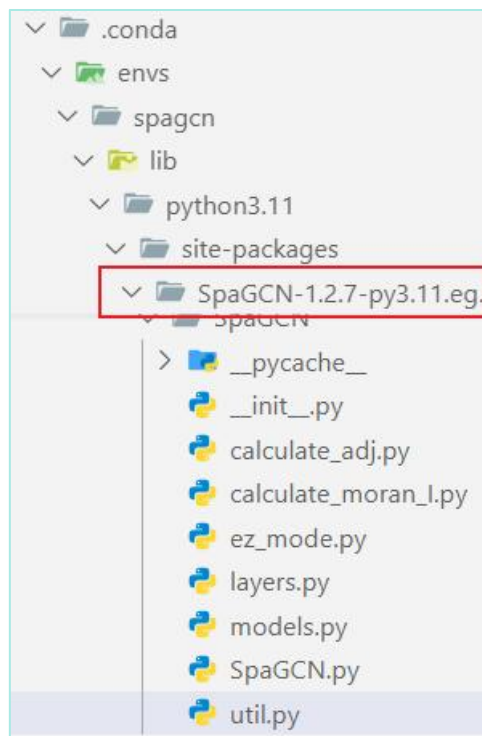
这个问题是因为我们在安装 github 包的时候,setup.py 文件没有添加这一行指令：

```

5
6  setuptools.setup(
7      name="SpaGCN",
8      version="1.2.7",
9      author="Jian Hu",
10     author_email="jianhu@penmedicine.upenn.edu",
11     description="SpaGCN: Integrating gene expression and histology to identify",
12     long_description=long_description,
13     long_description_content_type="text/markdown",
14     url="https://github.com/jianhuupenn/SpaGCN",
15     packages=setuptools.find_packages(),
16     install_requires=["python-igraph", "torch", "pandas", "numpy", "scipy", "scanpy",
17     #install_requires=[],
18     classifiers=[
19         "Programming Language :: Python :: 3",
20         "License :: OSI Approved :: MIT License",
21         "Operating System :: OS Independent",
22     ],
23     python_requires='>=3.6',
24     zip_safe = False,
25 )

```

现在将这个命令加进去，重新安装这个模块。（安装的方式说了很多遍了，这里省略）。再回来我们就可以编辑这个文件了：



2.13 生存分析

- TCGA (The Cancer Genome Atlas)
 - 数据内容: 乳腺癌患者的基因表达数据 (RNA-seq) 和临床生存数据 (生存时间、生存状态等)。
 - 如何获取:
 - 访问 [TCGA官网](#) 或通过工具 [UCSC Xena](#)、[cBioPortal](#)、[GEPIA2](#) 直接下载。
 - 在cBioPortal中搜索 “Breast Invasive Carcinoma (TCGA)”，选择包含临床信息的数据集。
 - 使用GEPIA2输入 “MALAT1” 和 “BRCA”，可快速生成生存曲线。
- GEO (Gene Expression Omnibus)
 - 数据内容: 公开的微阵列或RNA-seq数据集，需筛选包含乳腺癌患者生存信息的样本。
 - 如何获取:
 - 访问 [GEO官网](#)，搜索关键词 “breast cancer survival MALAT1” 或 “breast cancer expression clinical”。
 - 检查数据集描述是否包含生存时间和基因表达数据 (如GPL570平台)。
- KM Plotter
 - 工具链接: [kmplot.com](#)
 - 直接输入基因名 “MALAT1” 和癌症类型 “breast cancer”，可自动生成生存分析结果。

链接: <https://kmplot.com/analysis/>

直接输入想要的基因，设置参数即可绘图。

2.14 包的某个依赖已经被弃用解决方法

今天装 GeneClust 包的时候遇到这样一个问题：

```

error: subprocess-exited-with-error

x python setup.py egg_info did not run successfully.
  exit code: 1
  > [15 lines of output]
    The 'sklearn' PyPI package is deprecated, use 'scikit-learn'
    rather than 'sklearn' for pip commands.

    Here is how to fix this error in the main use cases:
    - use 'pip install scikit-learn' rather than 'pip install sklearn'
    - replace 'sklearn' by 'scikit-learn' in your pip requirements files
      (requirements.txt, setup.py, setup.cfg, Pipfile, etc ...)
    - if the 'sklearn' package is used by one of your dependencies,
      it would be great if you take some time to track which package uses
      'sklearn' instead of 'scikit-learn' and report it to their issue tracker
    - as a last resort, set the environment variable
      SKLEARN_ALLOW_DEPRECATED_SKLEARN_PACKAGE_INSTALL=True to avoid this error

    More information is available at
    https://github.com/scikit-learn/sklearn-pypi-package
    [end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
error: metadata-generation-failed
  
```

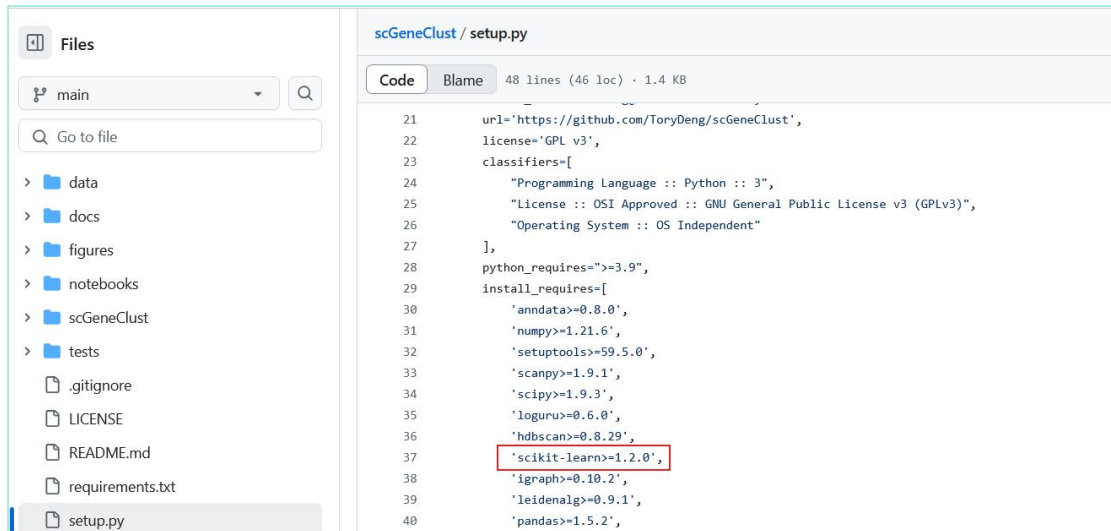
然后问了一下豆包，原来是 GeneClust 包的依赖 sklearn 已经被替换为替换成 scikit-learn：

从你给出的错误信息可知，在安装GeneClust时，它所依赖的sklearn包在 PyPI 上已经被弃用，现在应当使用 scikit-learn。下面为你提供几种解决办法：

方法一：联系包作者更新依赖

你可以向GeneClust的开发者反馈，让他们把setup.py或者requirements.txt里的sklearn替换成scikit-learn。

然后我们在 github 找到对应的源码，可以发现这里作者已经更正了，但是在 <https://pypi.org/> 的包应该并没有进行更正，所以此时我们需要手动安装。



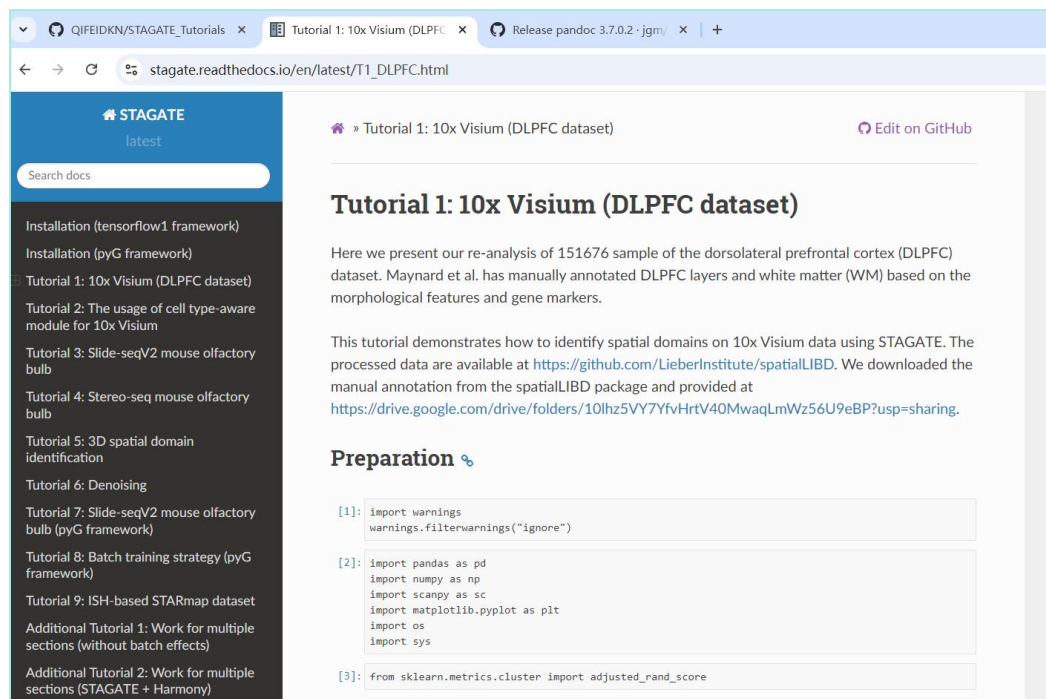
那么自己手动配置一下就好了。同时我遇到一个非常坑的问题，就是 `xrspatial` 实际应该是 `xarray-spatial`，哎搞了半天，还是 `deepseek` 告诉我的。

清除 pip 缓存: `pip cache purge`

2.15 Sphinx+github+ReadtheDocs 搭建自己的代码 Tutorials

目标链接: <https://stagate.readthedocs.io/en/latest/index.html>

对应的 github 链接: https://github.com/QIFEIDKN/STAGATE_Tutorials/tree/main



2.15.1 Sphinx 部分

如果你只想在网页上配置，请忽略终端代码 **Sphinx** 部分（本地部署）。

首先创建一个新的虚拟环境，然后安装 sphinx 一些必要的包：

```

pip install sphinx sphinx_rtd_theme nbsphinx jupyter sphinx-gallery
conda install -c conda-forge pandoc

```


然后建立一个新的文件夹，并 cd 进入：

```
(base) PS C:\Users\18085> conda activate sphinx
(sphinxx) PS C:\Users\18085> cd C:\Users\18085\Desktop\Test
(sphinxx) PS C:\Users\18085\Desktop\Test> ls
(sphinxx) PS C:\Users\18085\Desktop\Test> |
```

建一个 docs 文件夹用于后续操作：

```
(sphinxx) PS C:\Users\18085\Desktop\Test> mkdir docs

目录: C:\Users\18085\Desktop\Test

Mode                LastWriteTime         Length Name
----                -
d-----          2025/9/3   19:36         docs

(sphinxx) PS C:\Users\18085\Desktop\Test> ls

目录: C:\Users\18085\Desktop\Test

Mode                LastWriteTime         Length Name
----                -
d-----          2025/9/3   19:36         docs
```

接下来进行 sphinx 初始化：

```
(sphinxx) PS C:\Users\18085\Desktop\Test> cd docs
(sphinxx) PS C:\Users\18085\Desktop\Test\docs> sphinx-quickstart
Welcome to the Sphinx 8.1.3 quickstart utility.

Please enter values for the following settings (just press Enter to
accept a default value, if one is given in brackets).

Selected root path: .

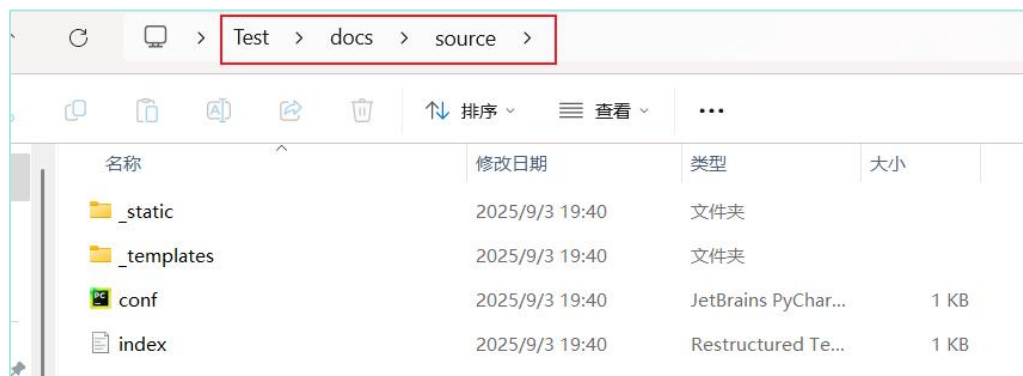
You have two options for placing the build directory for Sphinx output.
Either, you use a directory "_build" within the root path, or you separate
"source" and "build" directories within the root path.
> Separate source and build directories (y/n) [n]: y
```

下一步配置 Sphinx，运行 sphinx-quickstart 时，按以下建议配置：

分离源文件和构建目录：选择 y

- 项目名称：输入你的项目名称
- 作者名称：输入你的名字
- 项目版本：输入版本号 (如 0.1)
- 语言：zh_CN (如果需要中文文档)

然后配置文件就出来了：



接下来去配置 Sphinx 以支持 Jupyter 笔记本，我们需要去编辑配置文件，即打

开 docs/source/conf.py 文件，添加以下内容：

添加到扩展列表

```
extensions = [
    # 已有的扩展...
    'nbsphinx',
    'sphinx_gallery.load_style',
    'IPython.sphinxext.ipython_console_highlighting',
]
```

设置主题为 readthedocs 风格

```
html_theme = 'sphinx_rtd_theme'
```

配置 Jupyter 笔记本相关设置

```
nbsphinx_execute = 'auto' # 自动执行笔记本
```

```
nbsphinx_allow_errors = True # 允许执行错误
```



```
7 # https://www.sphinx-doc.org/en/master/usage/configuration.html#
8
9 project = 'FlatST'
10 copyright = '2025, cxd'
11 author = 'cxd'
12 release = '1.0.1'
13
14 # -- General configuration -----
15 # https://www.sphinx-doc.org/en/master/usage/configuration.html#
16
17 # extensions = []
18
19 templates_path = ['_templates']
20 exclude_patterns = []
21
22 # -- Options for HTML output -----
23 # https://www.sphinx-doc.org/en/master/usage/configuration.html#
24
25 html_theme = 'alabaster'
26 html_static_path = ['_static']
27 # 添加到扩展列表
28 extensions = [
29     # 已有的扩展...
30     'nbsphinx',
31     'sphinx_gallery.load_style',
32     'IPython.sphinxext.ipython_console_highlighting'
33 ]
34
35 # 设置主题为 readthedocs 风格
36 html_theme = 'sphinx_rtd_theme'
37 # 配置 Jupyter 笔记本相关设置
38 nbsphinx_execute = 'auto' # 自动执行笔记本
39 nbsphinx_allow_errors = True # 允许执行错误
```

下一步去创建索引文件，编辑 docs/source/index.rst，添加你的 Jupyter 笔记本文件（注意都是 3 个空格）：

```
.. toctree::
```

```
    :maxdepth: 2
```

:caption: Contents:

你的 jupyter 1

你的 jupyter 2

```
文件  编辑  查看

.. FlatST documentation master file, created by
   sphinx-quickstart on Wed Sep 3 19:40:54 2025.
   You can adapt this file completely to your liking, but it should at least
   contain the root `toctree` directive.

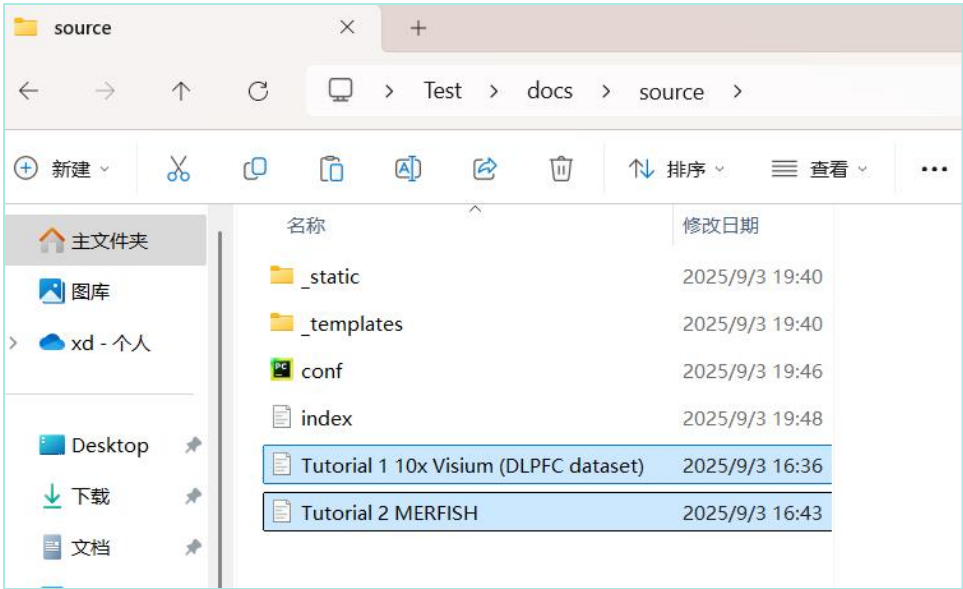
FlatST documentation
=====

Add your content using ``reStructuredText`` syntax. See the
`reStructuredText <https://www.sphinx-doc.org/en/master/usage/restructuredtext/index.html>`_
documentation for details.

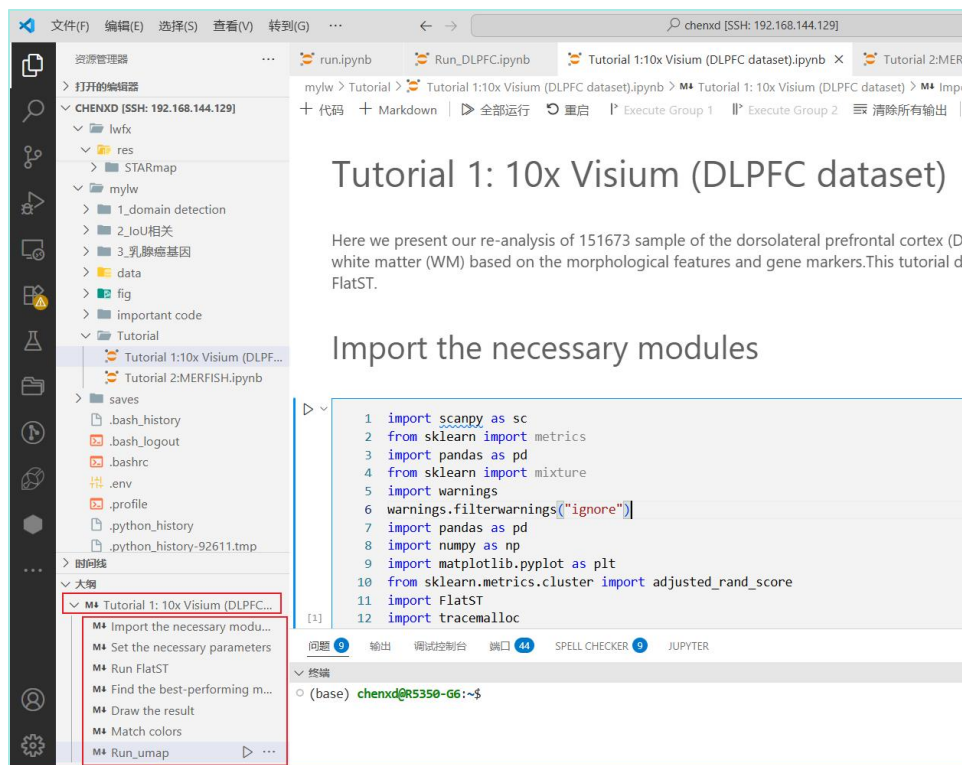
.. toctree::
   :maxdepth: 2
   :caption: Contents:

   Tutorial 1 10x Visium (DLPFC dataset)
   Tutorial 2 MERFISH
```

把你准备好的 jupyter 文档复制到当前目录下：



推荐 jupyter 文档是一个一级目录和多个二级目录：



然后就可以生成 HTML 文档了，在 docs 目录下运行 make html:

```
(sphinx) PS C:\Users\18085\Desktop\Test\docs> make html
Running Sphinx v8.1.3
loading translations [en]... done
loading pickled environment... The configuration has changed (4 options: 'html_perma...
hinx_requirejs_options', 'nbsphinx_requirejs_path')
done
Writing evaluated template result to C:\Users\18085\Desktop\Test\docs\build\html\_static\nbsphinx-code-cells.css
building [mo]: targets for 0 po files that are out of date
writing output...
building [html]: targets for 0 source files that are out of date
updating environment: 2 added, 1 changed, 0 removed
reading sources... [100%] index
looking for now-outdated files... none found
pickling environment... done
checking consistency... done
preparing documents... done
```

如果遇到报错可以尝试下列命令，没报错则跳过:

.\make.bat html

接下来就可以去查看生成的文档，生成的 HTML 文件位于 docs/build/html 目录下，打开 index.html 即可查看。

FlatST

Search docs

CONTENTS:

- Tutorial 1: 10x Visium (DLPFC dataset)
 - Import the necessary modules
 - Set the necessary parameters
 - Run FlatST
 - Find the best-performing model
 - Draw the result
 - Match colors
 - Run_umap
- Tutorial 2: MERFISH dataset

/ Tutorial 1: 10x Visium (DLPFC dataset) [View page source](#)

Tutorial 1: 10x Visium (DLPFC dataset)

Here we present our re-analysis of 151673 sample of the dorsolateral prefrontal cortex (DLPFC) dataset. Maynard et al. has manually annotated DLPFC layers and white matter (WM) based on the morphological features and gene markers. This tutorial demonstrates how to identify spatial domains on 10x Visium data using FlatST.

Import the necessary modules

```
[1]: import scanpy as sc
from sklearn import metrics
import pandas as pd
from sklearn import mixture
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.cluster import adjusted_rand_score
import FlatST
import tracemalloc
import time
from sklearn.metrics.cluster import normalized_mutual_info_score
```

Set the necessary parameters

```
[2]: # Set parameters
number_list = [151507, 151508, 151509, 151510, 151516, 151670, 151671, 151672, 151673, 151674, 151675]
```

2.15.2 github 部分

请注意，每个.ipynb 文件至少要用 markdown 做一个标题，最好 1 个一级标题和多个二级标题构成，因为一级标题就是你显示的目录。

上述我们的本地端就设置好了，下面我们讲解如何进行网页端部署，那样所有人都能看到这个页面了。下面我们需要创建一个新的 github 仓库来线上保存我们生成的内容，方便我们以后对页面进行管理。取一个自己的名字，然后其他默认即可。

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * CHENszu / FlatST_Test
 FlatST_Test is available.

Great repository names are short and memorable. How about **musical-spork**?

Description
 0 / 350 characters

2 Configuration

Choose visibility * Public
 Choose who can see and commit to this repository

Add README Off
 READMEs can be used as longer descriptions. [About READMEs](#)

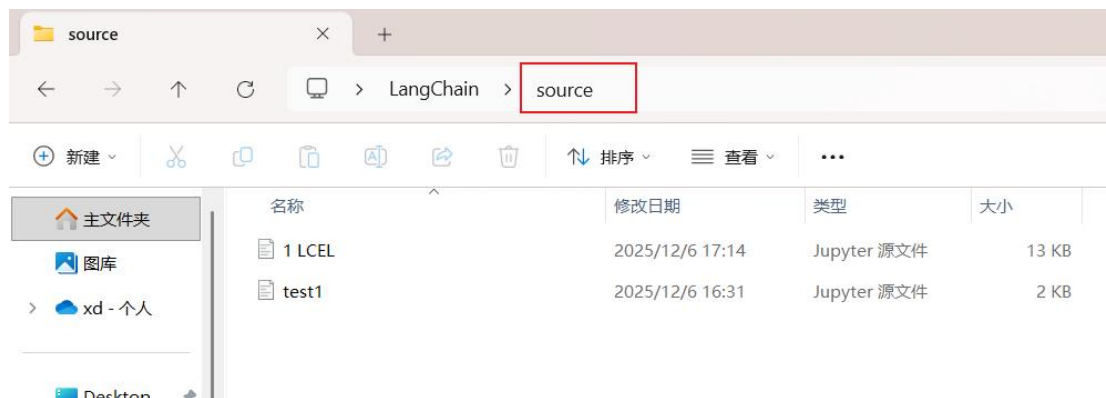
Add .gitignore No .gitignore
 .gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
 Licenses explain how others can use your code. [About licenses](#)

Create repository

首先要确保你的服务器能够访问 github, 可以通过 `ping github.com` 或者 `nslookup github.com` 来看看能否正常访问 github, 不行就在自己本地进行。

请注意文件格式应该是这样的, 你要上传的 `ipynb` 文件在 `source` 文件夹下面:



打开命令行 (CMD 或 PowerShell), 进入目标文件夹:

```
cd C:\Users\18085\Desktop\LangChain
```

初始化本地仓库:

```
git init
```

重命名分支:

```
git branch -m master main
```

配置你的 github 信息:

```
git config user.email "你的邮箱@xxx.com"
```

```
git config user.name "你的用户名"
```

将文件添加到暂存区:

```
git add .
```

提交文件到本地仓库:

```
git commit -m "这里写提交说明"
```

配置仓库:

```
git remote add origin https://github.com/你的用户名/test1.git
```

检查是否配置成功:

```
git remote -v
```

关联远程仓库, 如果之前关联过其他仓库 (没有不用管), 先执行 `git remote rm origin` 再重新关联即可:

```
git remote add origin https://github.com/CHENsuzu/FlatST_Tutorial.git
```

推送文件到远程仓库:

```
git push -u origin main
```

(如果远程默认分支是 `master`, 把 `main` 换成 `master`。

```
cd C:\Users\18085\Desktop\Test\docs
```

```
echo "# t" >> README.md
```

git add README.md

git init

git add .

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/CHENszu/t.git

git push -u origin main

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

加载个人及系统配置文件用了 2345 毫秒。
(base) PS C:\Users\18085\Desktop\Test\docs> git init
Reinitialized existing Git repository in C:/Users/18085/Desktop/Test/docs/.git/
(base) PS C:\Users\18085\Desktop\Test\docs> git add .
(base) PS C:\Users\18085\Desktop\Test\docs> git commit -m "first commit"
On branch master
nothing to commit, working tree clean
(base) PS C:\Users\18085\Desktop\Test\docs> git branch -M main
(base) PS C:\Users\18085\Desktop\Test\docs> git remote add origin https://github.com/CHENszu/t.git
error: remote origin already exists.
(base) PS C:\Users\18085\Desktop\Test\docs> git remote rm origin
(base) PS C:\Users\18085\Desktop\Test\docs> git remote add origin https://github.com/CHENszu/t.git
(base) PS C:\Users\18085\Desktop\Test\docs> git push -u origin main
Enumerating objects: 101, done.
Counting objects: 100% (101/101), done.
Delta compression using up to 16 threads
Compressing objects: 100% (101/101), done.
Writing objects: 100% (101/101), 7.44 MiB | 567.00 KiB/s, done.
Total 101 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), done.
To https://github.com/CHENszu/t.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(base) PS C:\Users\18085\Desktop\Test\docs>
```

如果报下列错误（不报错请跳过）：

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

加载个人及系统配置文件用了 2353 毫秒。
(base) PS C:\Users\18085\Desktop\Test\docs> git init
Reinitialized existing Git repository in C:/Users/18085/Desktop/Test/docs/.git/
(base) PS C:\Users\18085\Desktop\Test\docs> git add .
(base) PS C:\Users\18085\Desktop\Test\docs> git commit -m "update docs files"
On branch main
nothing to commit, working tree clean
(base) PS C:\Users\18085\Desktop\Test\docs> git remote rm origin
(base) PS C:\Users\18085\Desktop\Test\docs> git remote add origin https://github.com/CHENszu/FlatST_Test.git
(base) PS C:\Users\18085\Desktop\Test\docs> git push -u origin main
To https://github.com/CHENszu/FlatST_Test.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/CHENszu/FlatST_Test.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

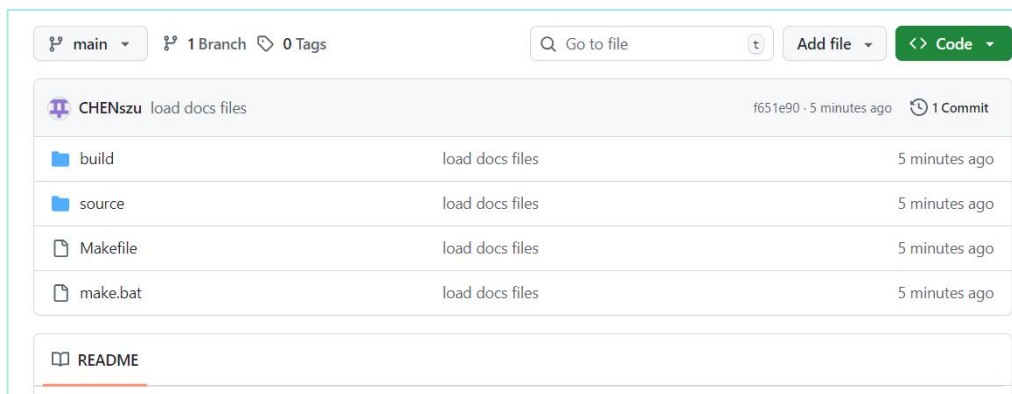
先把远程仓库的最新状态拉到本地：

git pull origin main --allow-unrelated-histories

（这一步会弹出编辑器让你写合并信息，直接保存退出即可）然后再次推送：

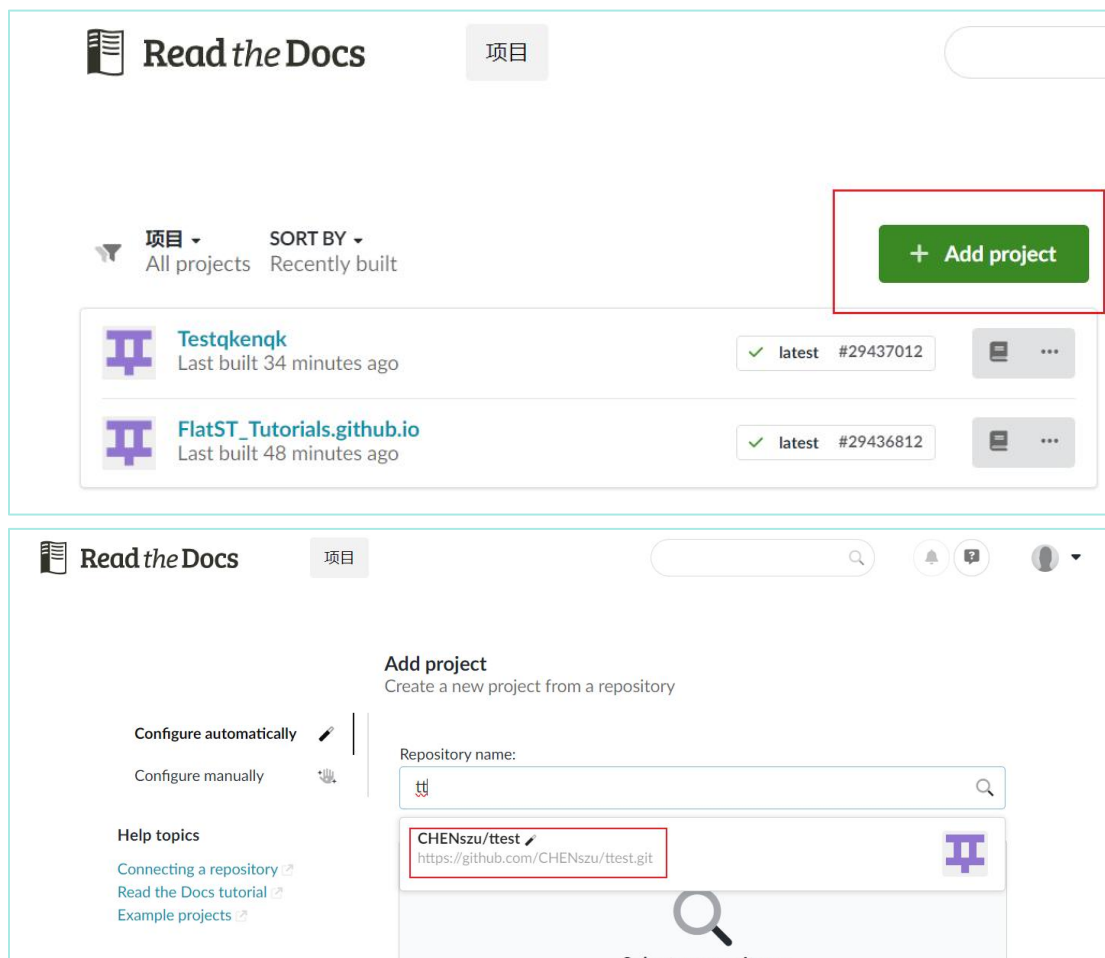
git push -u origin main

然后我们的文件就正常上传了：



2.15.3 ReadtheDocs 部分

官方网址：<https://app.readthedocs.org/> 然后使用 github 登录，其就可以自动获取我们仓库的 Repositories，具体操作步骤为：



Add project

Configure basic project settings

Help topics

- [Connecting a repository](#)
- [Read the Docs tutorial](#)
- [Example projects](#)

名称 *

代码库地址 ? *

默认分支 ?

语言 ? *

English

下一页

系统提示缺少这个文件，那么我们复制下来去 github 补充：

Add project

Add a configuration file to your project

Help topics

- [Configuration file tutorial](#)
- [Configuration file reference](#)
- [Manually configuring a Git repository](#)

A **.readthedocs.yaml** file is required at the root of your repository to build your project's documentation. You can pick an example for your documentation tool as a starting point below, and save and commit it to your repository.

Example configuration for: **Sphinx**

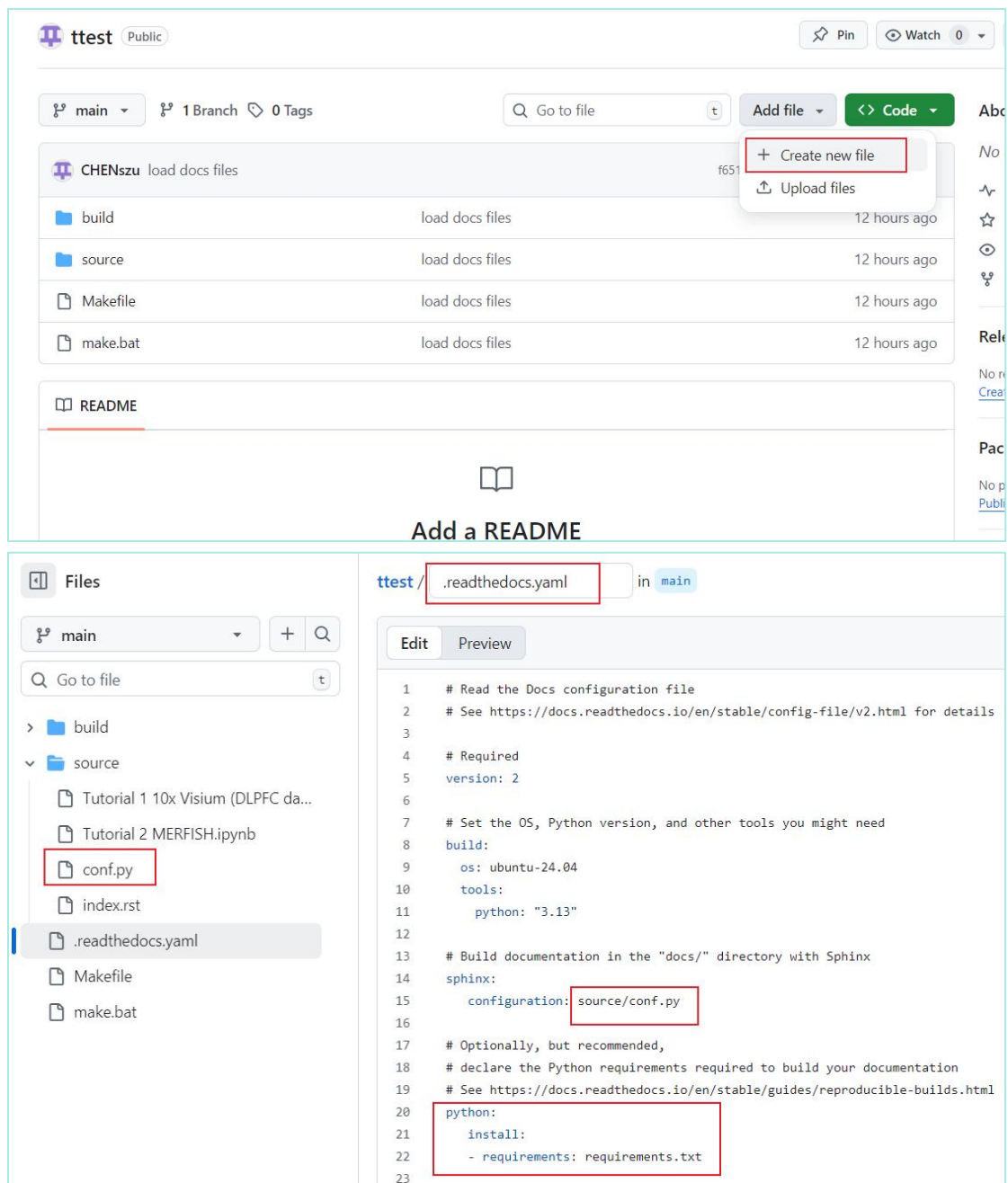
```
.readthedocs.yaml
# Read the Docs configuration file
# See https://docs.readthedocs.io/en/stable/config-file/v2.html for details

# Required
version: 2

# Set the OS, Python version, and other tools you might need
build:
  os: ubuntu-24.04
  tools:
    python: "3.13"
```

上一页 I need help This file exists

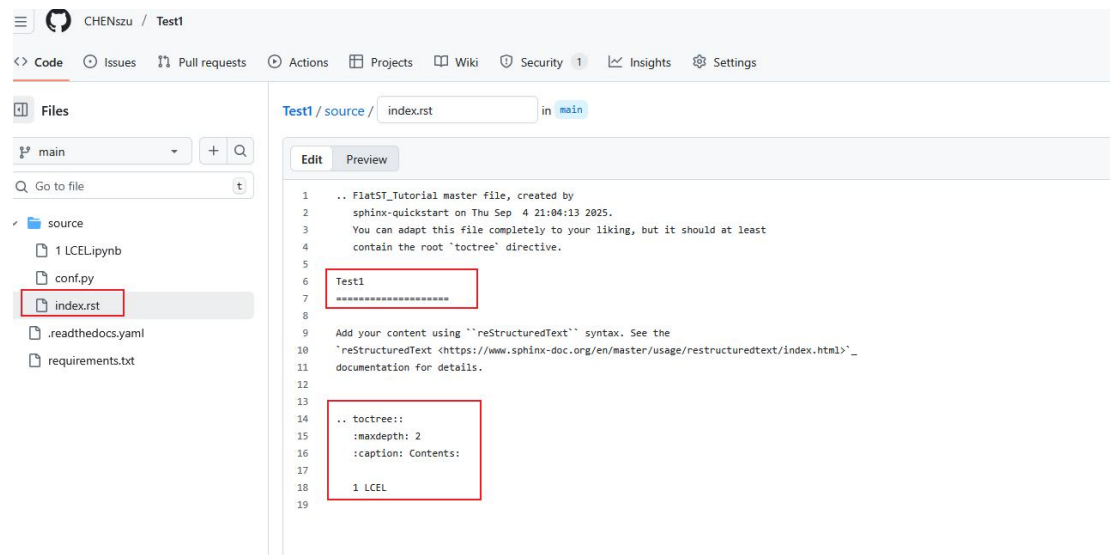
注意最下面那 3 行 requirements.txt 一定要取消注释：



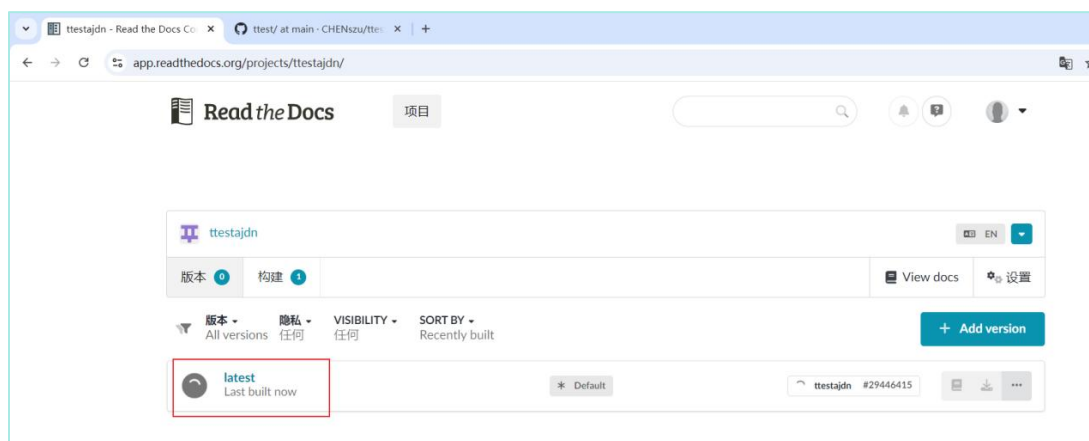
The screenshot displays a GitHub repository interface for a project named 'ttest'. The repository is public and has one branch, 'main', with no tags. The file list shows a directory structure with 'build' and 'source' folders, and files 'Makefile' and 'make.bat'. A 'README' section is highlighted, and a large 'Add a README' button is visible. Below this, the file editor for '.readthedocs.yaml' is shown, containing a configuration snippet for Read the Docs. The snippet includes comments and configuration details for Sphinx, such as the OS, Python version, and the location of the configuration file.

```
1 # Read the Docs configuration file
2 # See https://docs.readthedocs.io/en/stable/config-file/v2.html for details
3
4 # Required
5 version: 2
6
7 # Set the OS, Python version, and other tools you might need
8 build:
9   os: ubuntu-24.04
10  tools:
11    python: "3.13"
12
13 # Build documentation in the "docs/" directory with Sphinx
14 sphinx:
15   configuration: source/conf.py
16
17 # Optionally, but recommended,
18 # declare the Python requirements required to build your documentation
19 # See https://docs.readthedocs.io/en/stable/guides/reproducible-builds.html
20 python:
21   install:
22     - requirements: requirements.txt
23
```

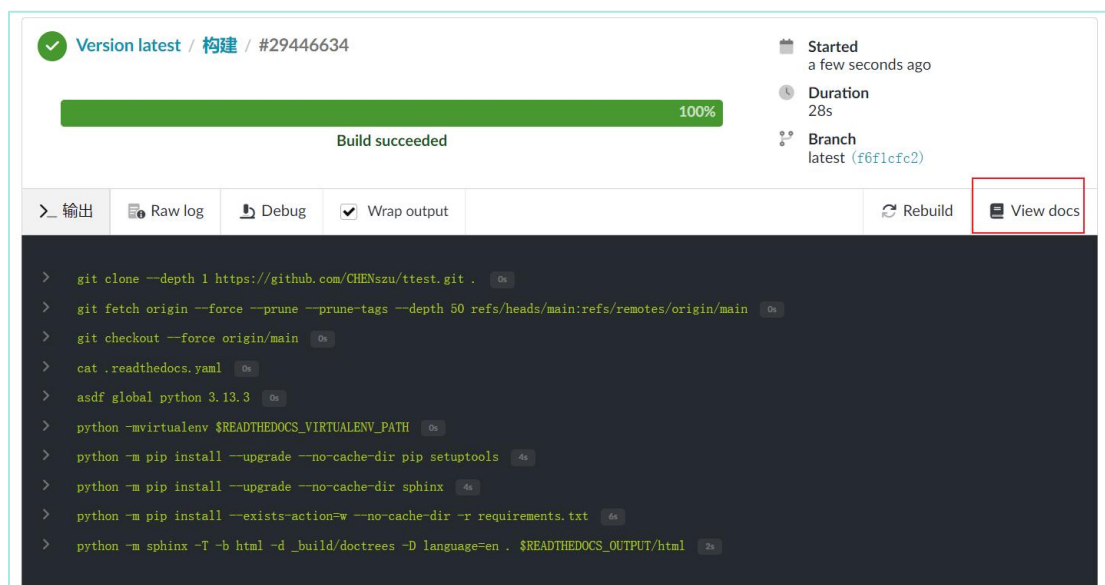
然后我们需要补充 `conf.py`，里面的内容可以复制我的然后修改自己的参数即可
https://github.com/CHENszu/FlatST_Tutorial/blob/main/source/conf.py



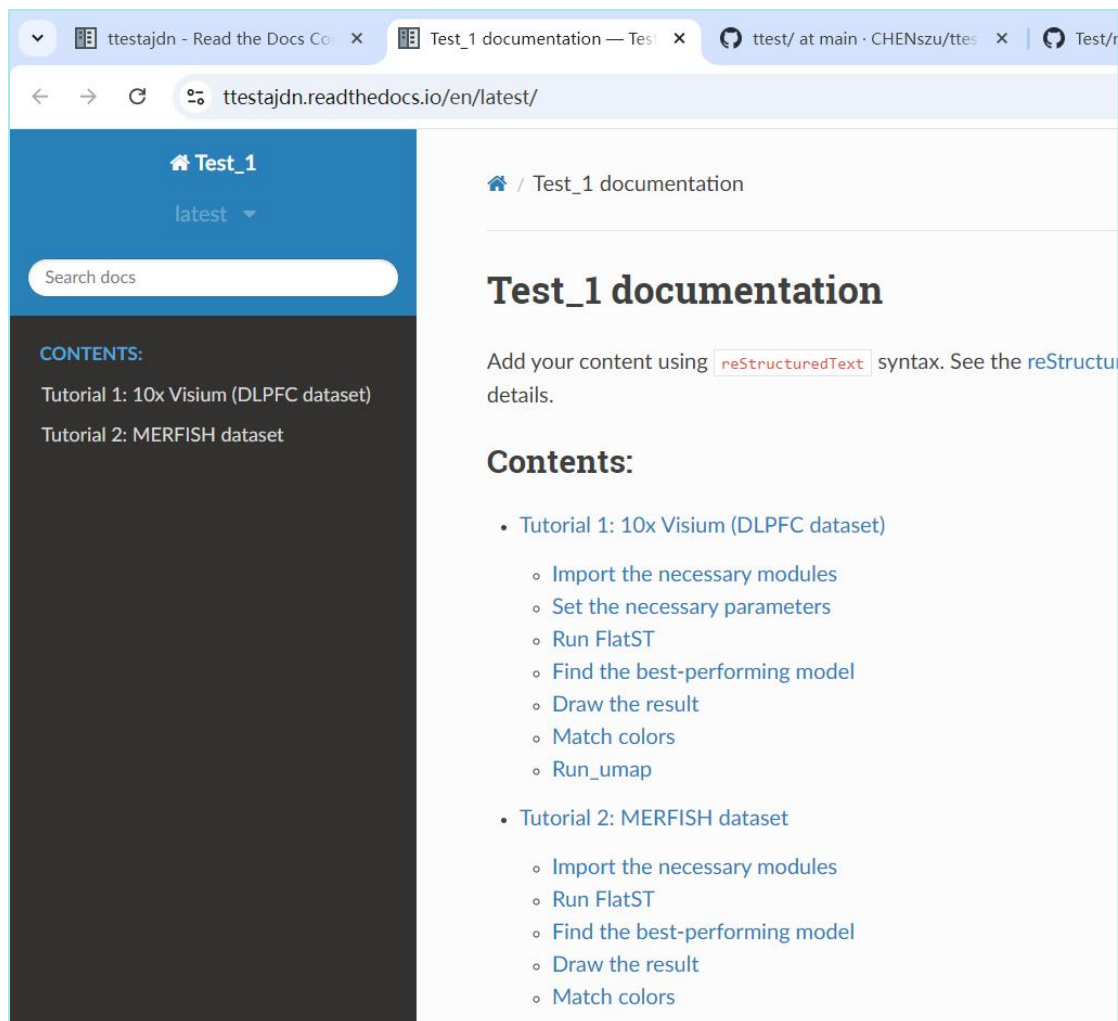
然后我们重新回到并 build 项目，点击红色边框查看详情：



可以发现已经构建成功了，点击即可打开你的网页链接，如果失败了请根据报错提示去修改 github 相关组件。



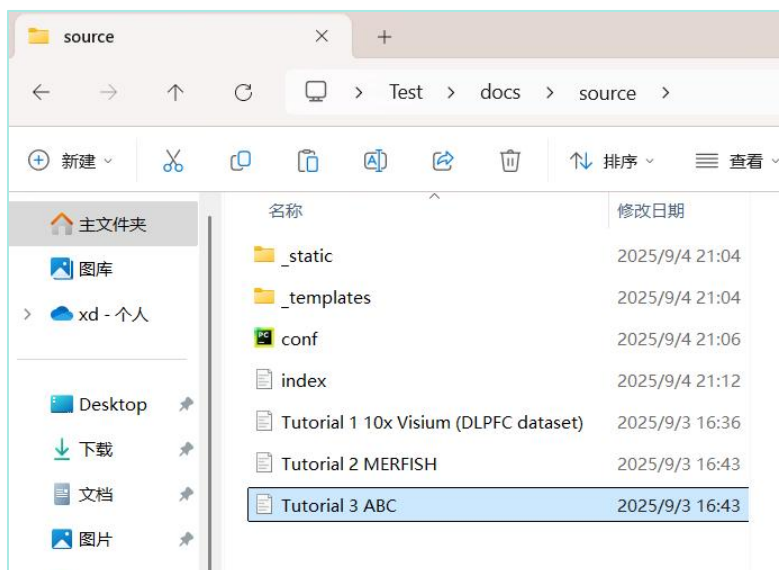
最后所有人都能轻松访问这个链接了：



<https://ttestajdn.readthedocs.io/en/latest/> 复制去任何浏览器都能打开（项目删除则链接失效）。

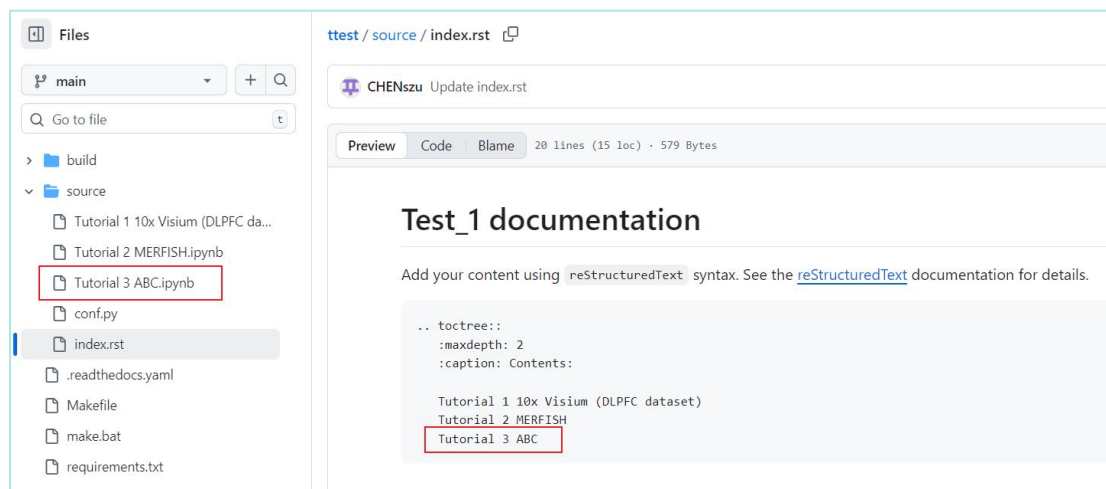
2.15.4 如何去更新网站

接下来讲讲我们后面想要在搭建好的网站增加 jupyter notebook 的做法。

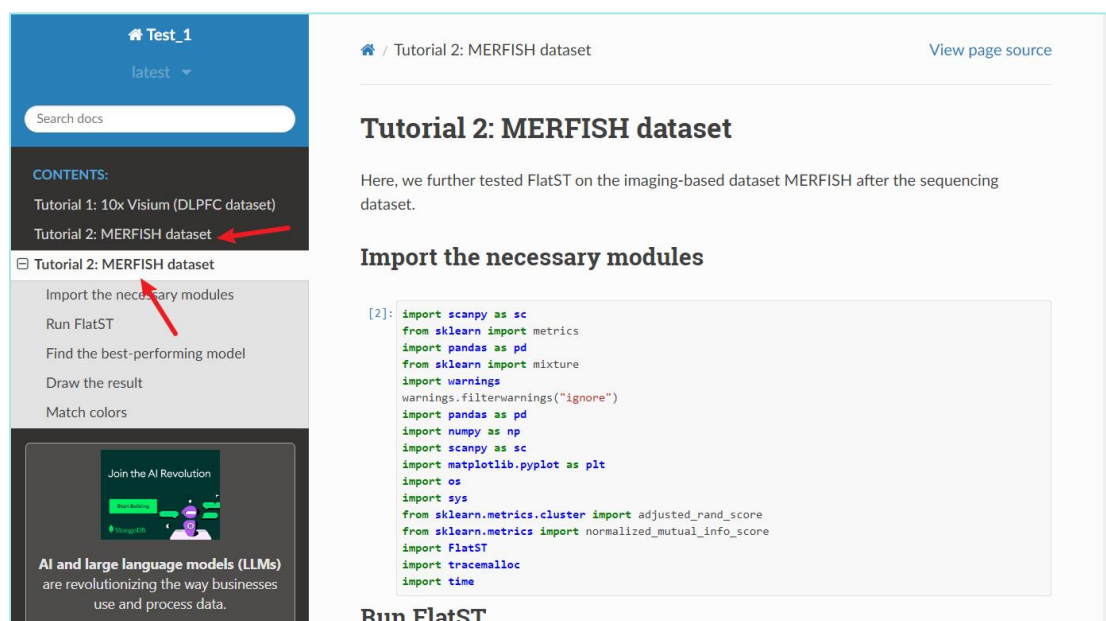


假设 Tutorial 3 ABC 是我们更新的 jupyter 文件，我这里图简单，直接把第二个

文件复制过来了。然后我们将其上传到 `github` 和其他文件一样的地方，然后更新 `index.rst` 文件：



然后刷新一下 ReadtheDocs 的项目，可以看到新的项目就出来了，为什么重名？这是因为这里的名字是按照 `jupyter` 的一级目录和二级目录来的，并不是按照上述文件名称来的。



下列代码不用管，仅做存档记录。

进入项目目录

```
cd /c/Users/18085/Desktop/Test
```

初始化 Git 仓库（如果尚未初始化）

```
git init
```

配置用户信息（全局设置，只需执行一次）

```
git config --global user.email "你的邮箱@example.com"
```

```
git config --global user.name "你的名字"
```

```
# 关联远程仓库（只需执行一次）
git remote add origin https://github.com/CHENszu/FlatST_Tutorial.git
# 添加当前目录所有文件
git add .
# 提交文件
git commit -m "上传 Test 目录下的所有文件"
# 推送到远程仓库（注意分支名称一致性）
# 查看远程仓库的默认分支是 master 还是 main
git remote show origin
# 根据实际分支名称推送，例如如果是 main 分支
git push -u origin main

# 将本地 master 分支重命名为 main
git branch -m master main

# 推送并重命名远程分支
git push -u origin main

# 如果需要删除远程的 master 分支
git push origin --delete master
```

安装 ghp-import:

```
pip install ghp-import
```

部署文档，在项目根目录运行：

```
ghp-import -n -p -f ./build/html
```

该命令会完全更新覆盖 docs/build/html 目录中的所有文件（HTML、CSS、JavaScript、图片等静态资源）

2.16 vscode 使用 jupyter 时无法选择虚拟环境

我们 jupyter 无法显示 python 环境，这个时候一般是 jupyter 拓展的问题，我们换一个版本安装就好了。

